

# Projeto e Análise de Algoritmos

Introdução à Grafos

Thiago Pinheiro de Araújo

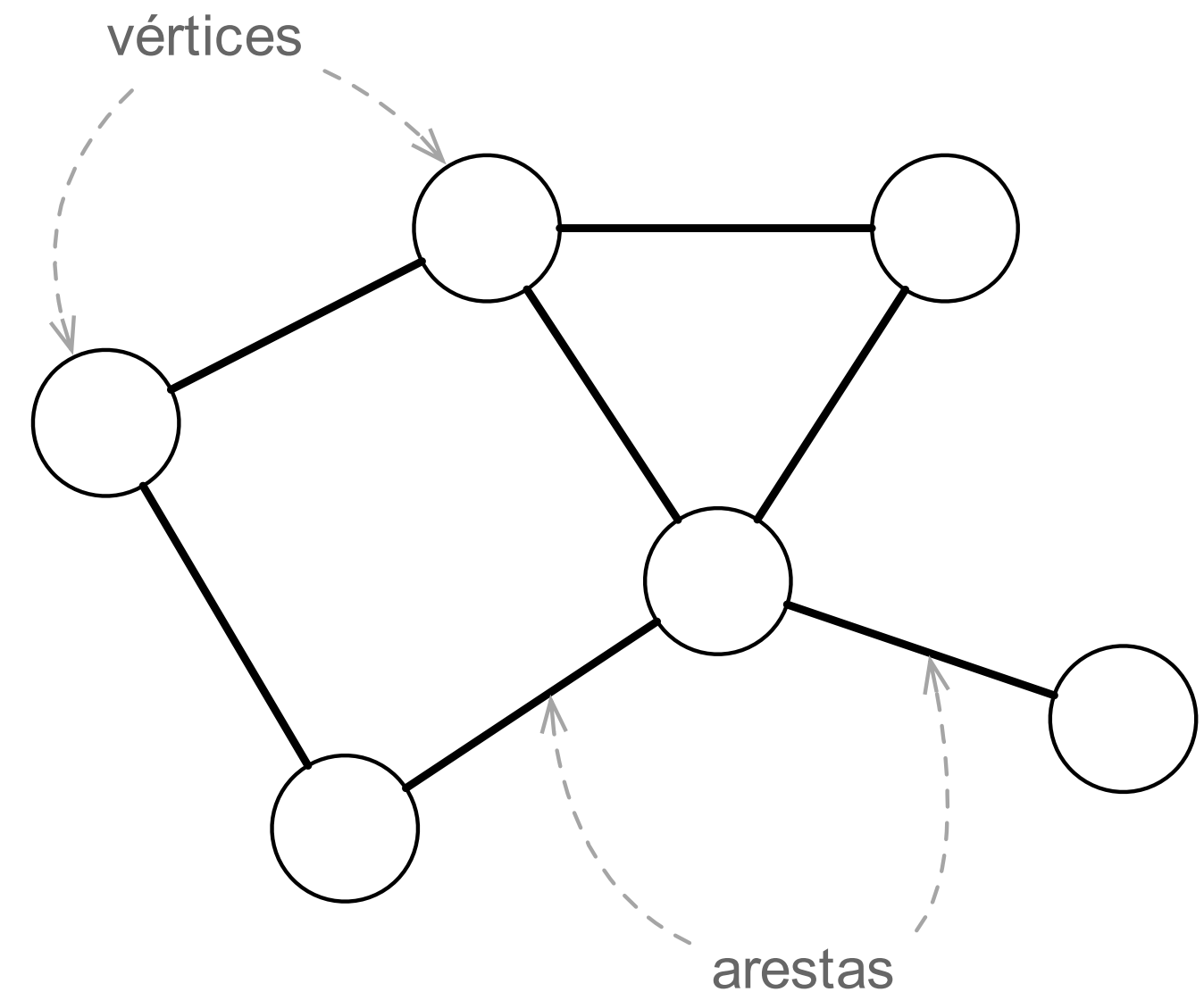
EMAp/FGV

2025.2

Apresentar a definição de **grafos**, suas variações, características e estratégias de implementação.

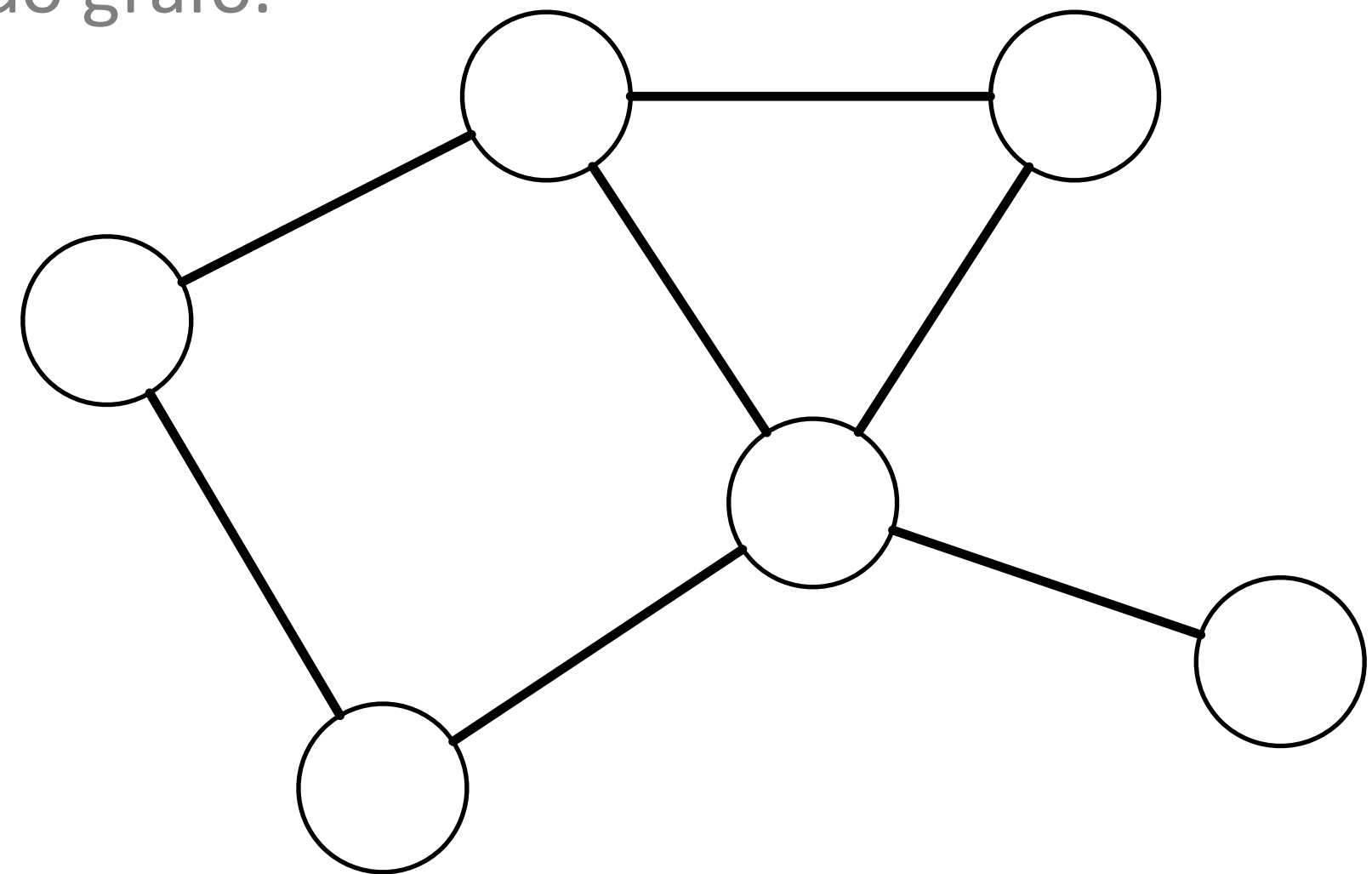
## Definição

- Um **grafo** é uma estrutura abstrata capaz de representar um conjunto de elementos e as relações entre eles.
- Esses elementos são chamados de vértices (V) e as relações de arestas (E)
  - Vértices também costumam ser chamados de nós.
- Uma aresta é definida por um par de vértices  $(v_i, v_j)$ .



## Definição

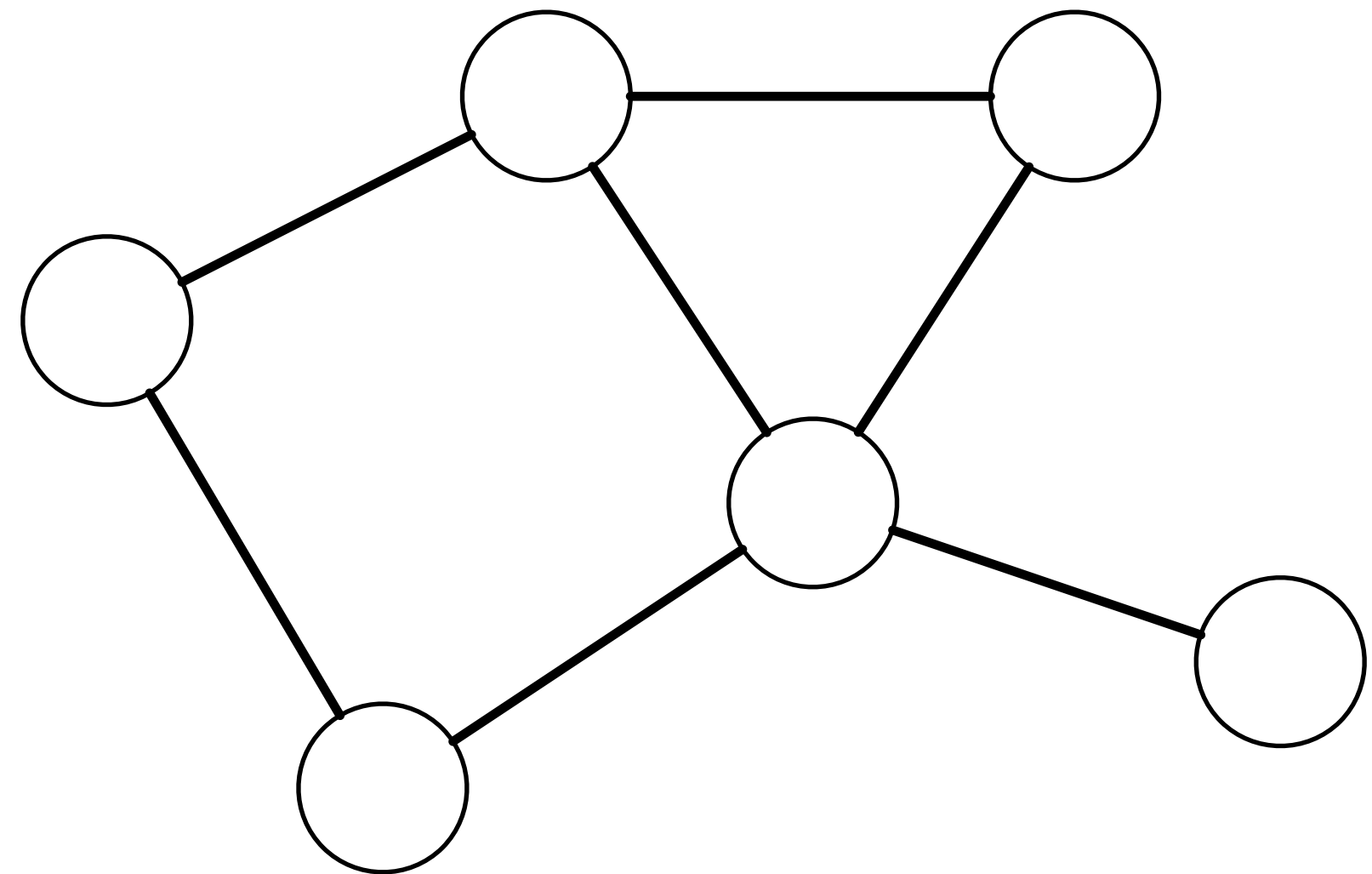
- Vamos representar um grafo por  $G = (V, E)$ 
  - $V$  corresponde ao número de vértices  $|V|$  do grafo.
  - $E$  corresponde ao número de arestas  $|E|$  do grafo.
- Sempre iremos considerar ambos os parâmetros  $(V, E)$  na análise de um algoritmo sobre um grafo
  - Ex:  $O(V + E)$ .



# Introdução à Grafos

## Motivação

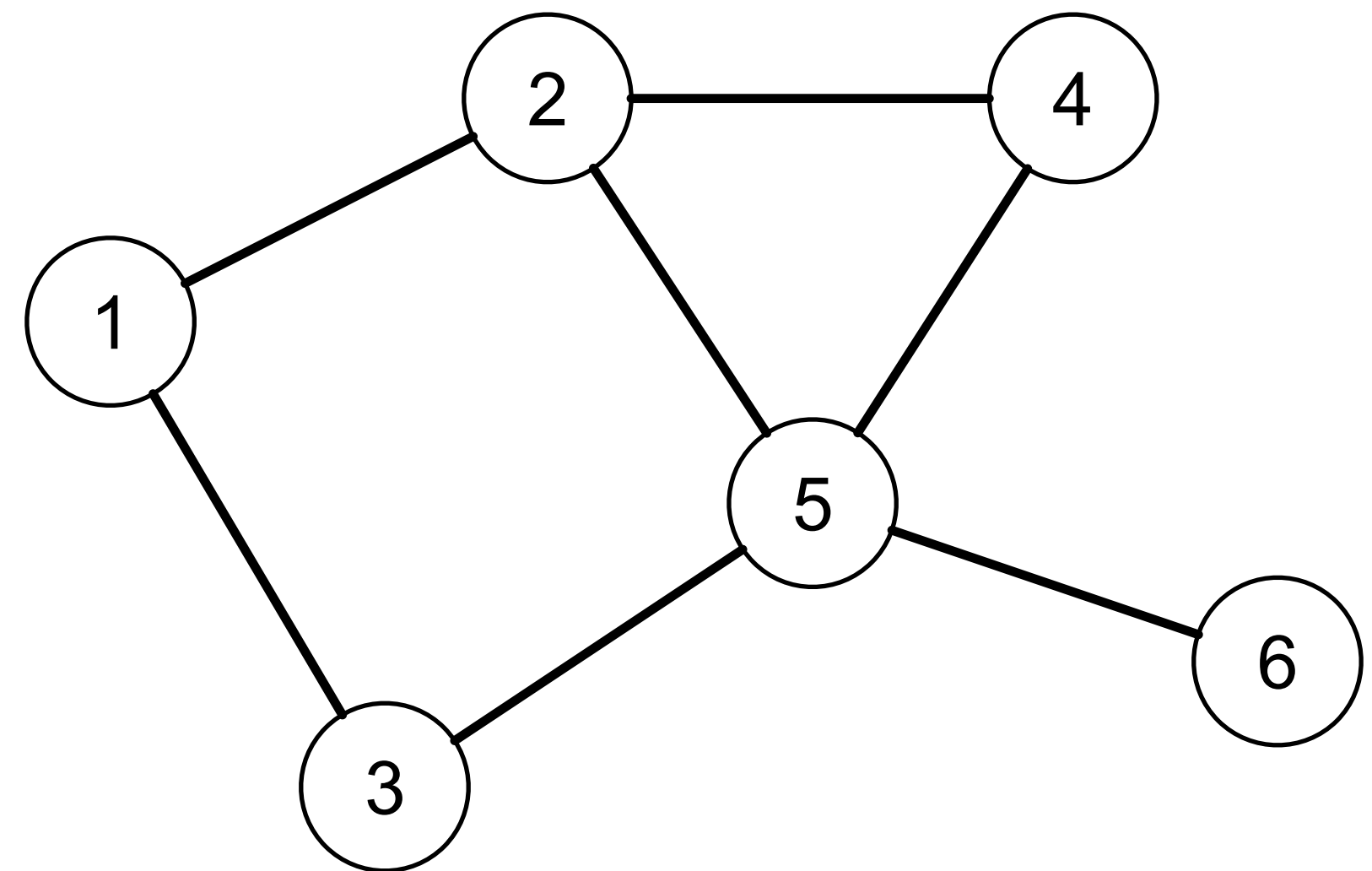
- Grafos e algoritmos sobre eles são utilizados para **modelar e resolver** diversos problemas do mundo real.
- Alguns exemplos:
  - Tráfego em uma cidade.
  - Definição de rotas em um mapa.
  - Compra de passagens aéreas.
  - Redes de computadores.
  - Garbage collector.
  - Redes sociais.
  - Etc.





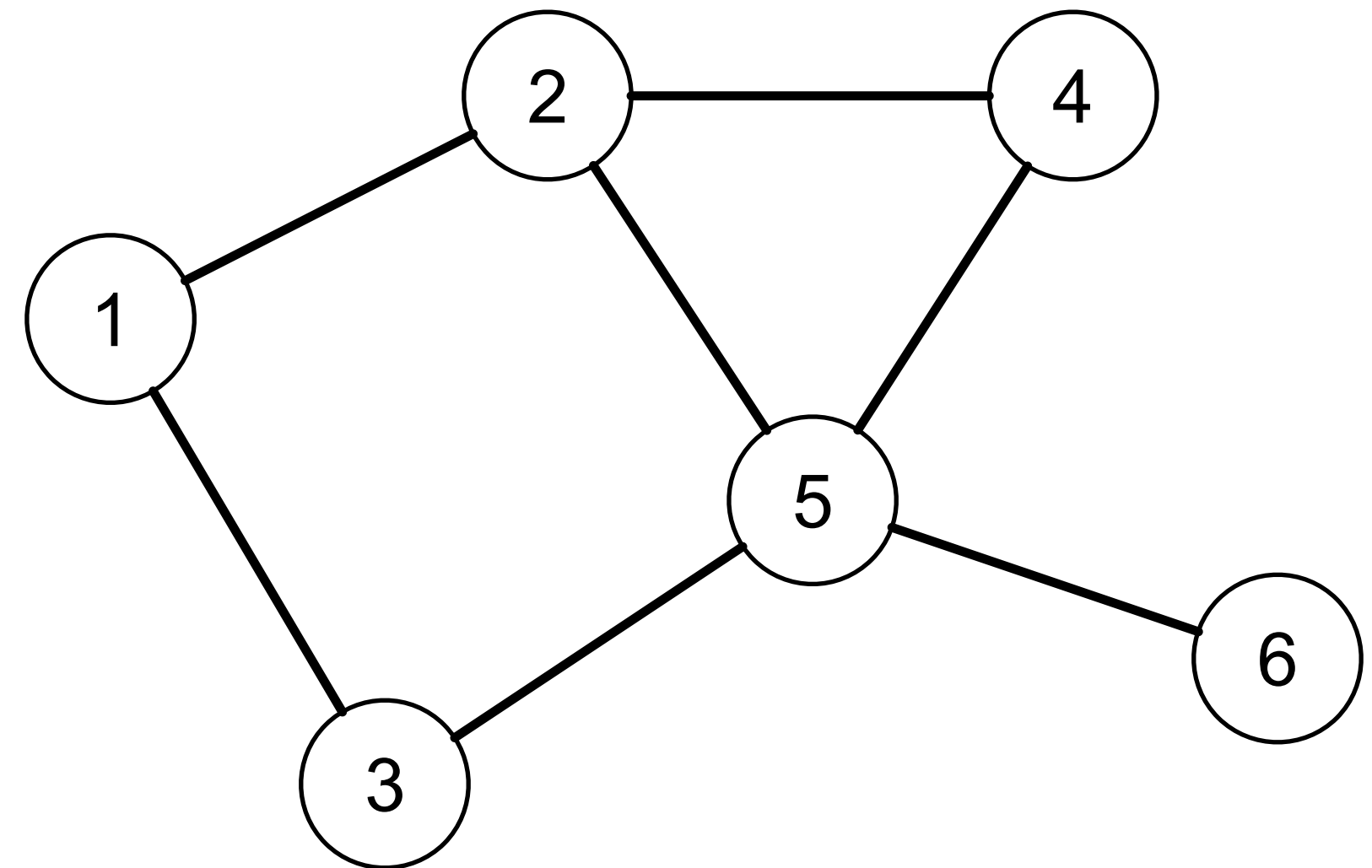
## Representação

- É comum identificar os vértices para trabalhar sobre a estrutura do grafo.
- Dessa forma podemos representar um grafo por:
  - $V = \{1,2,3,4,5,6\}$
  - $E = \{(1,2), (1,3), (2,4), (2,5), (3,5), (4,5), (5,6)\}$
- O tamanho de um grafo é definido por  $|V| + |E|$ .



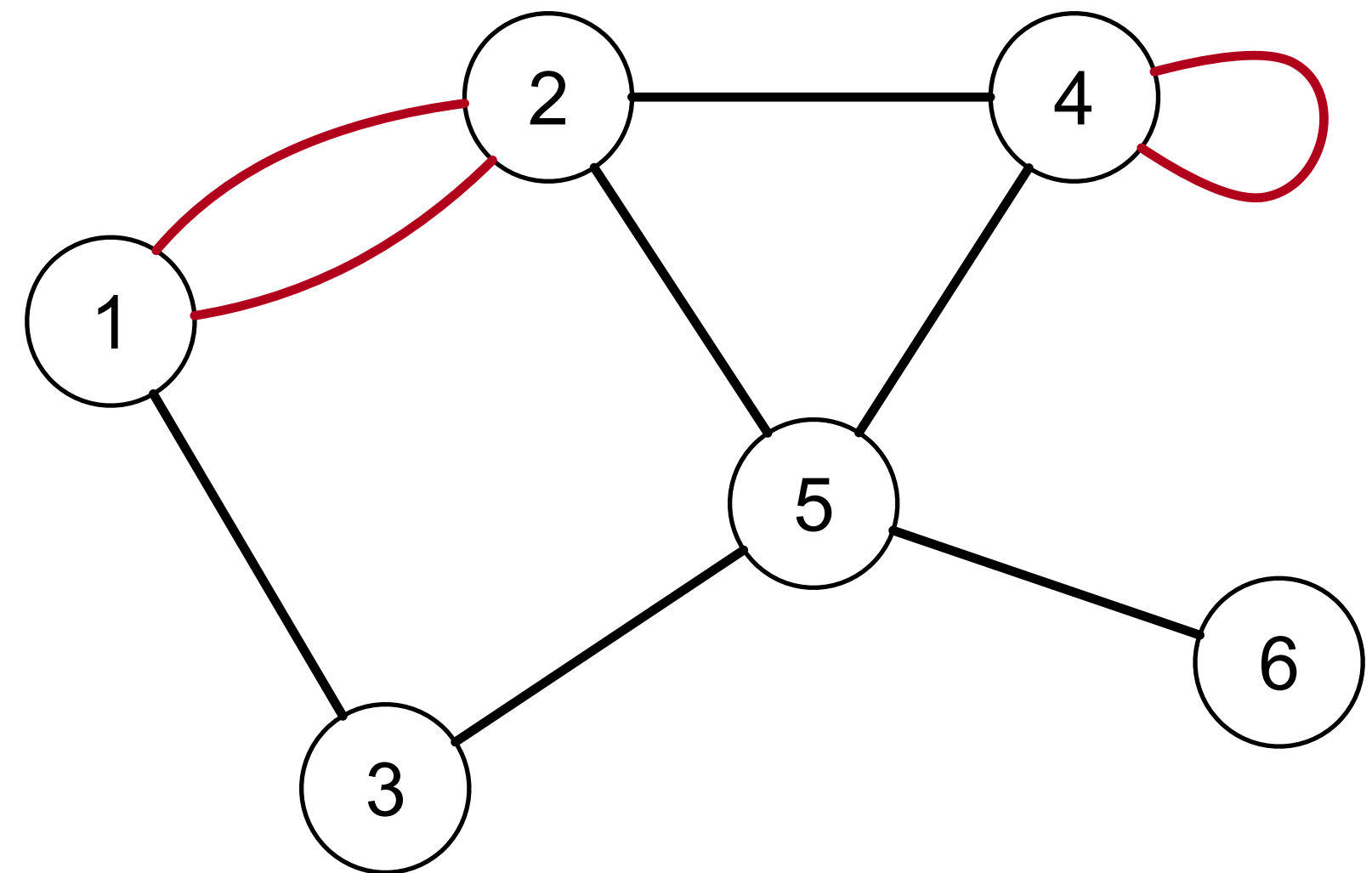
## Adjacência

- Dada uma aresta  $e = (v_i, v_j)$  podemos dizer que os vértices  $v_i$  e  $v_j$  são os extremos da aresta.
- Também pode-se dizer que a aresta  $e$  é incidente aos vértices  $v_i$  e  $v_j$  e esses são incidentes à aresta  $e$ .
- Por fim, também dizemos que vértices relacionados por uma aresta são adjacentes.



## Adjacência

- Duas arestas  $e_1$  e  $e_2$  são consideradas paralelas se forem incidentes aos mesmos vértices  $(v_i, v_j)$ .
- Um laço é uma aresta incidente a um único vértice  $e = (v_i, v_i)$ .
- Exceto quando definido explicitamente assumiremos que o grafo de entrada não possui laços ou arestas paralelas.





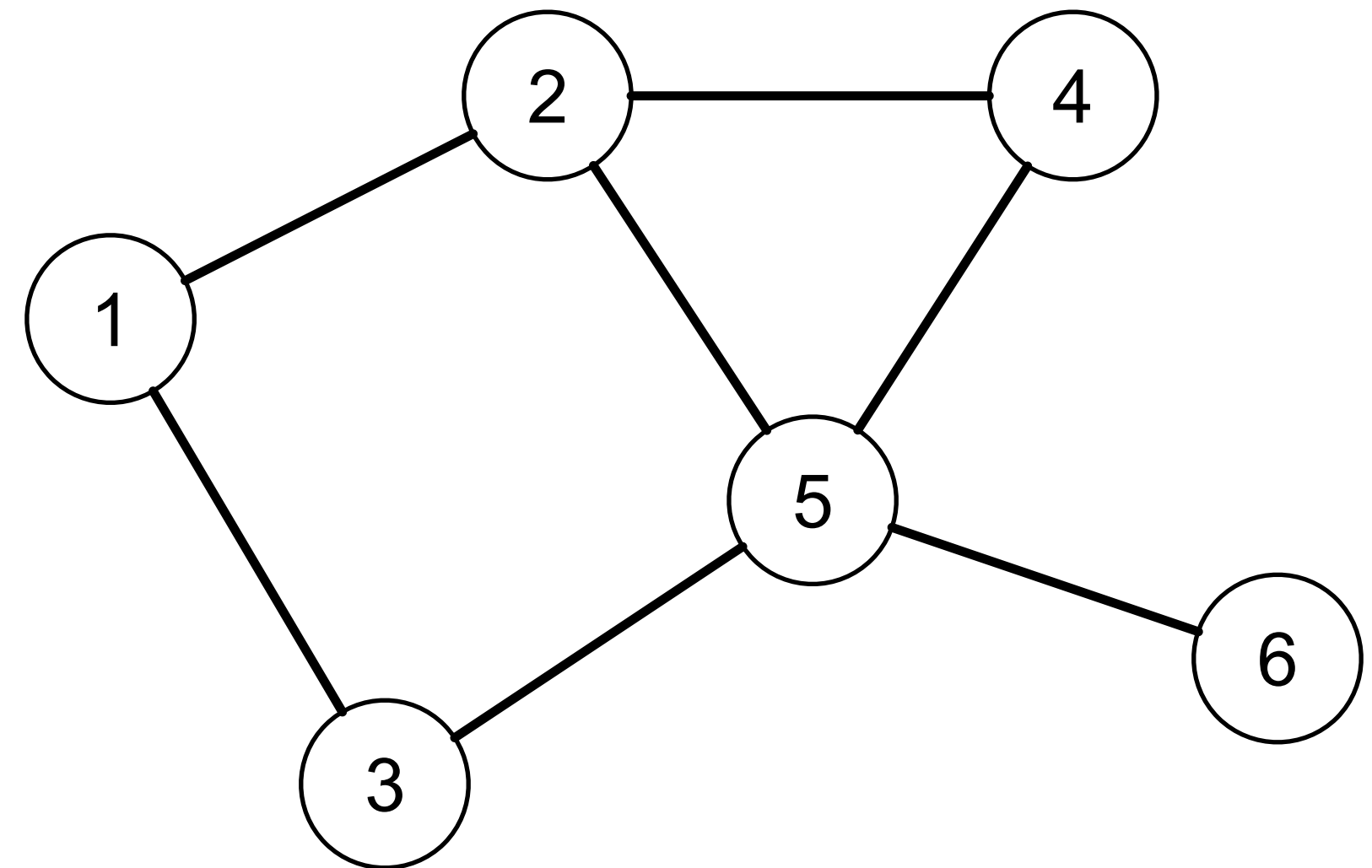
- O **grau**  $g(v_i)$  de um vértice  $v_i$  é definido pelo número de vértices adjacentes à ele.

- Para o grafo ao lado temos:

$$\begin{array}{ll} g(1) = 2 & g(4) = 2 \\ g(2) = 3 & g(5) = 4 \\ g(3) = 2 & g(6) = 1 \end{array}$$

- A soma dos graus de todos os vértices é igual ao dobro do número de arestas

$$\sum_{i=1}^{|V|} g(v_i) = 2|E|$$

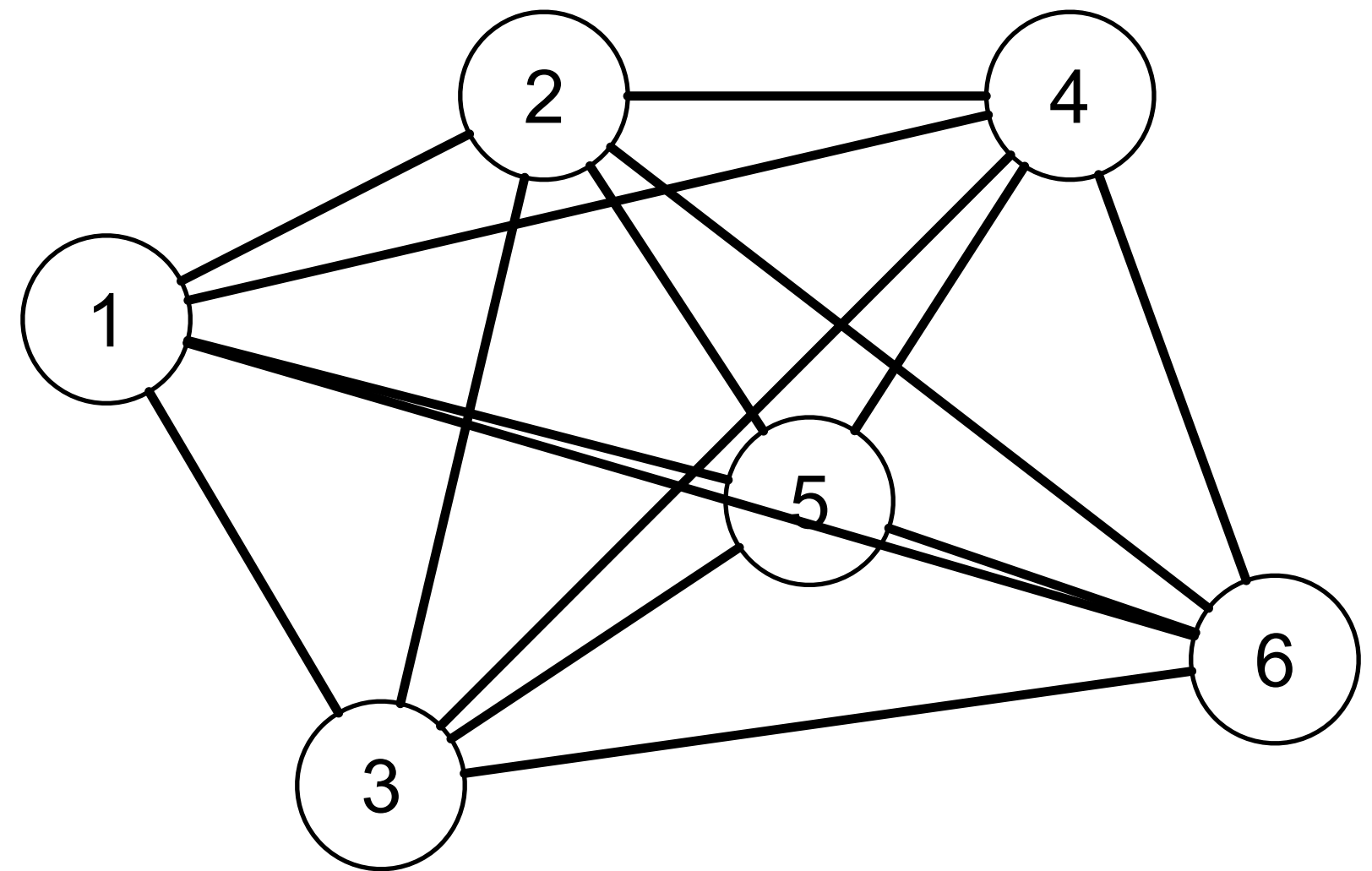


- Um grafo é completo se cada vértice possuir todos os demais adjacentes à ele.

- O número de arestas em um grafo completo é definido por:

$$\frac{|V|(|V| - 1)}{2}$$

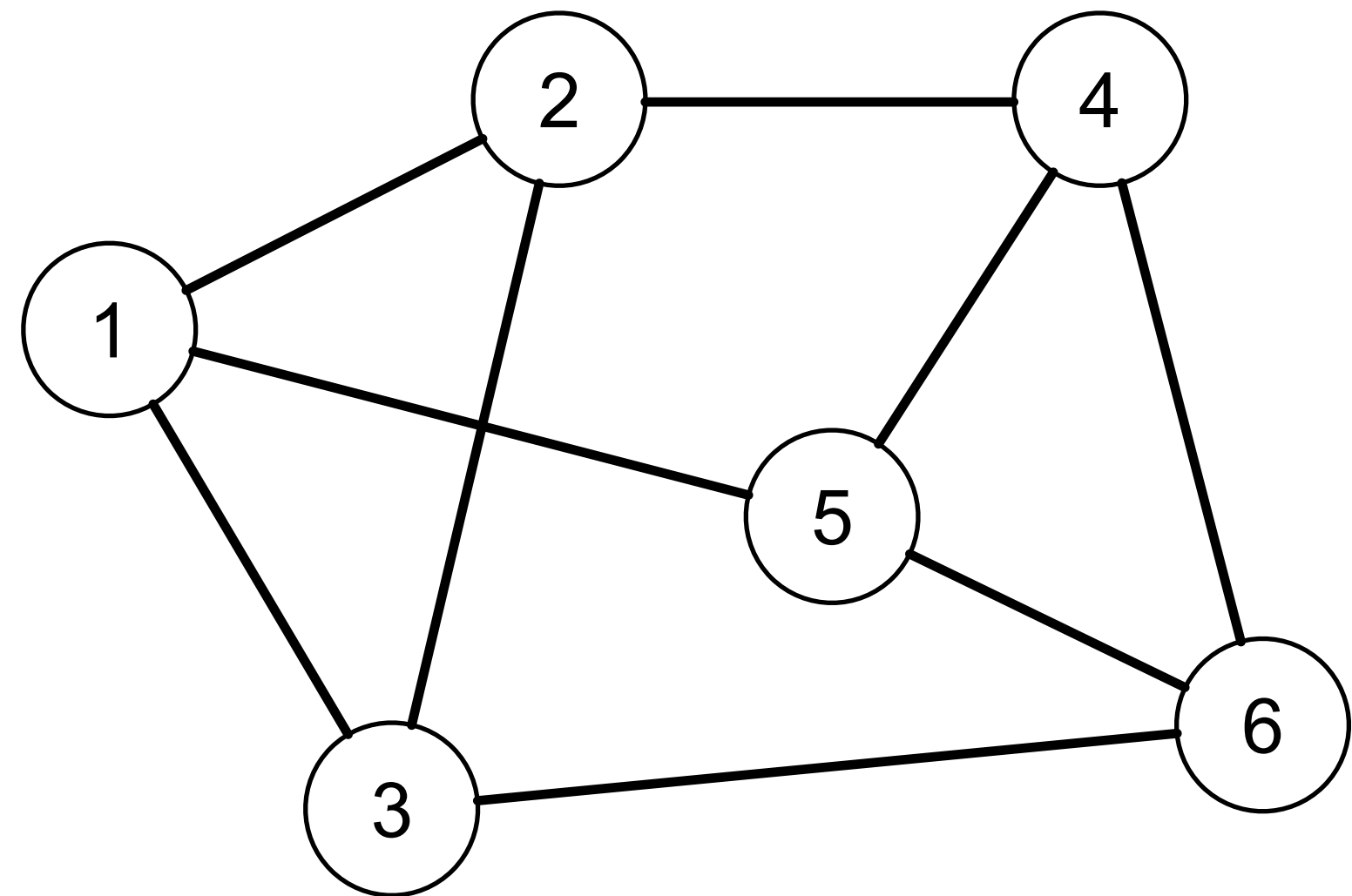
- Portanto, observe que o número máximo de arestas em um grafo  $G = (V, E)$  é ligeiramente menor que  $|V|^2/2$ .



## Características

- Um grafo é regular se todos os vértices possuírem o mesmo grau.
- Dizemos que um grafo é k-regular, definindo k como seu grau:
  - O grafo ao lado é 3-regular.
- O número de arestas de um grafo regular é definido por:

$$\frac{|V|r}{2}$$



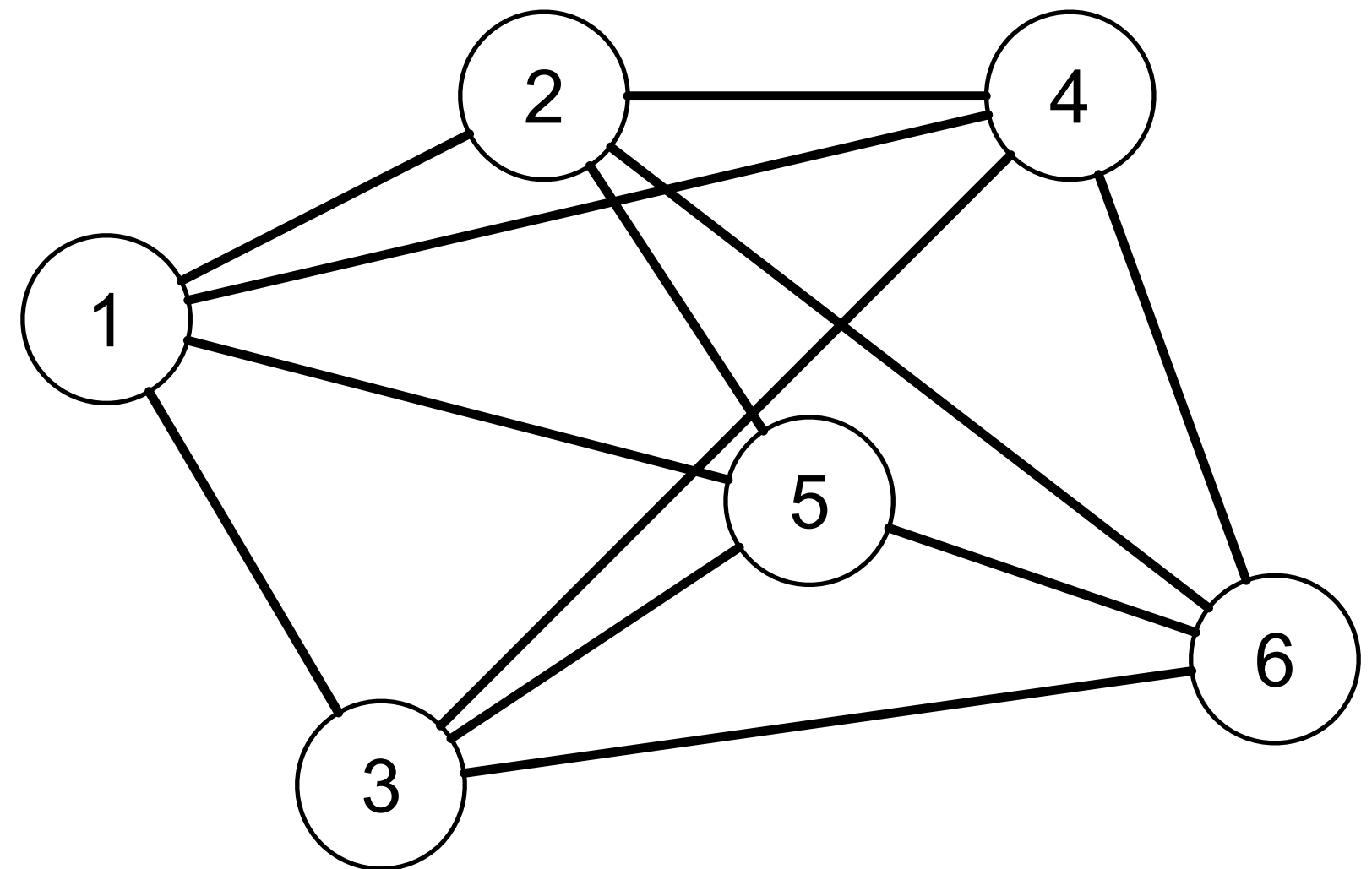
## Características

- Um grafo é denso se o seu tamanho for proporcional ao quadrado do número de vértices

$$(|V| + |E|) \propto |V|^2$$

- Um grafo é esparso se o seu tamanho for proporcional ao número de vértices

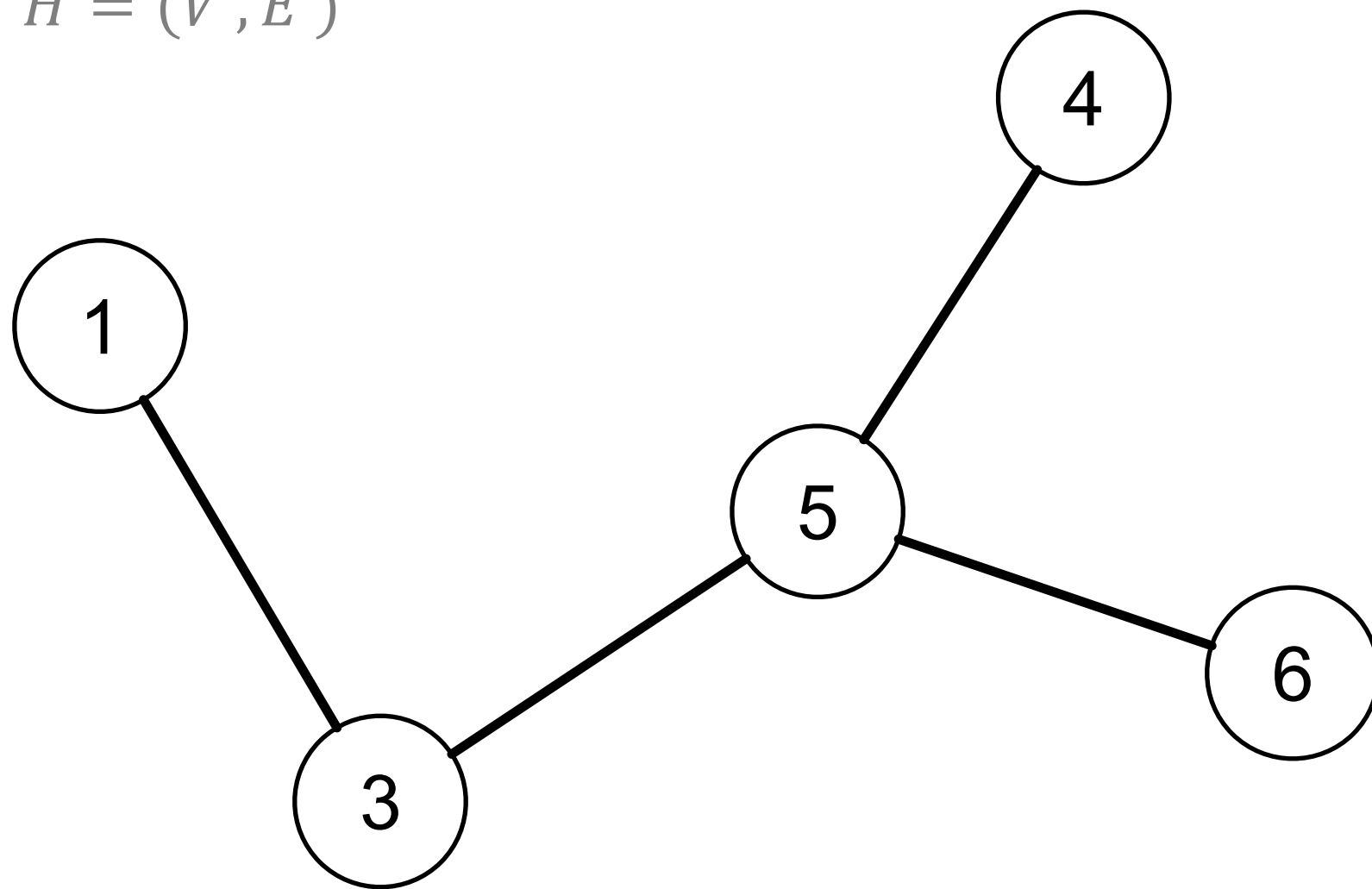
$$(|V| + |E|) \propto |V|$$



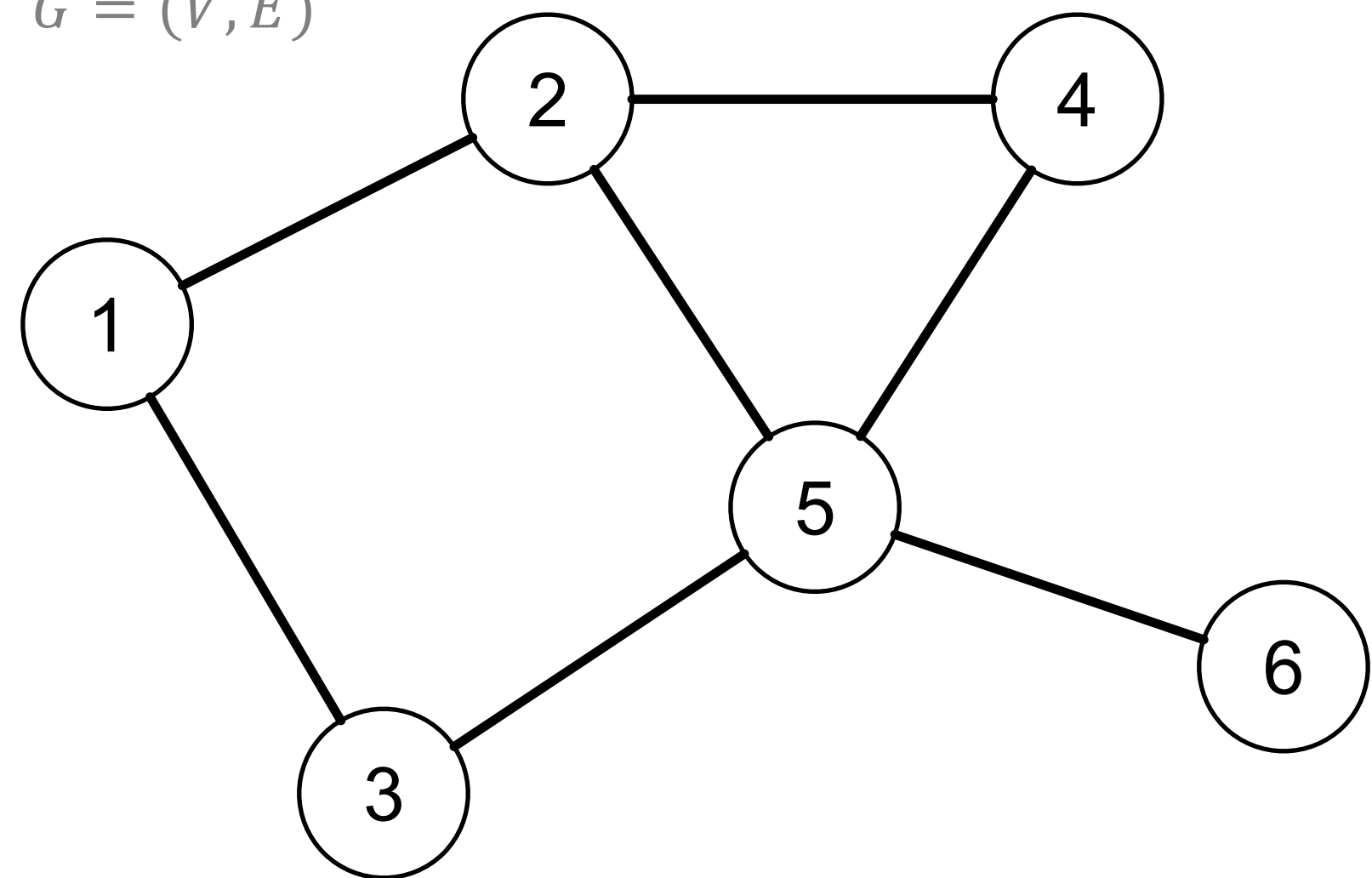
## Subgrafos

- O grafo  $H = (V', E')$  é um **subgrafo** de  $G = (V, E)$  se  $V' \subseteq V$  e  $E' \subseteq E$ .

$H = (V', E')$



$G = (V, E)$

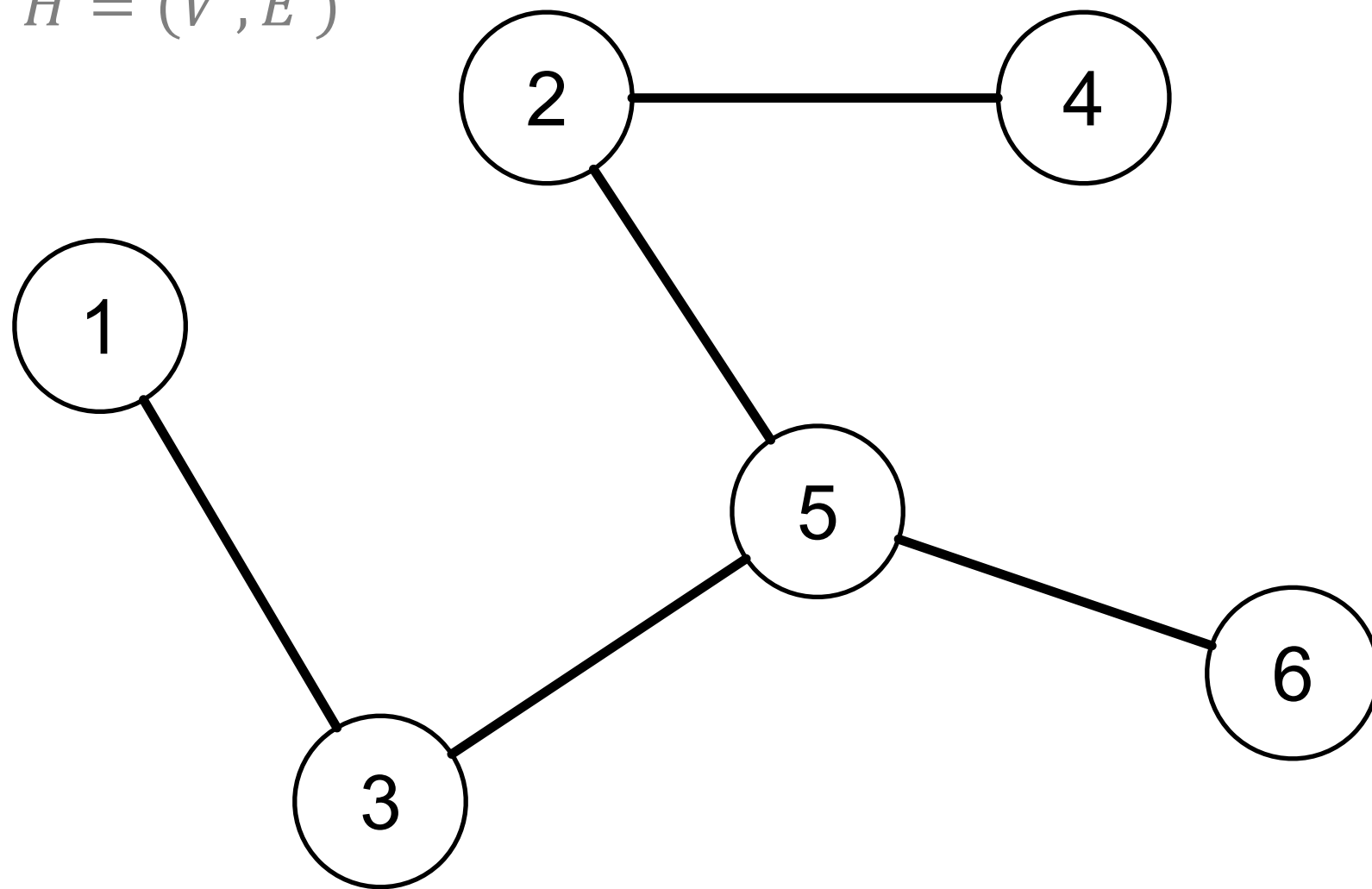




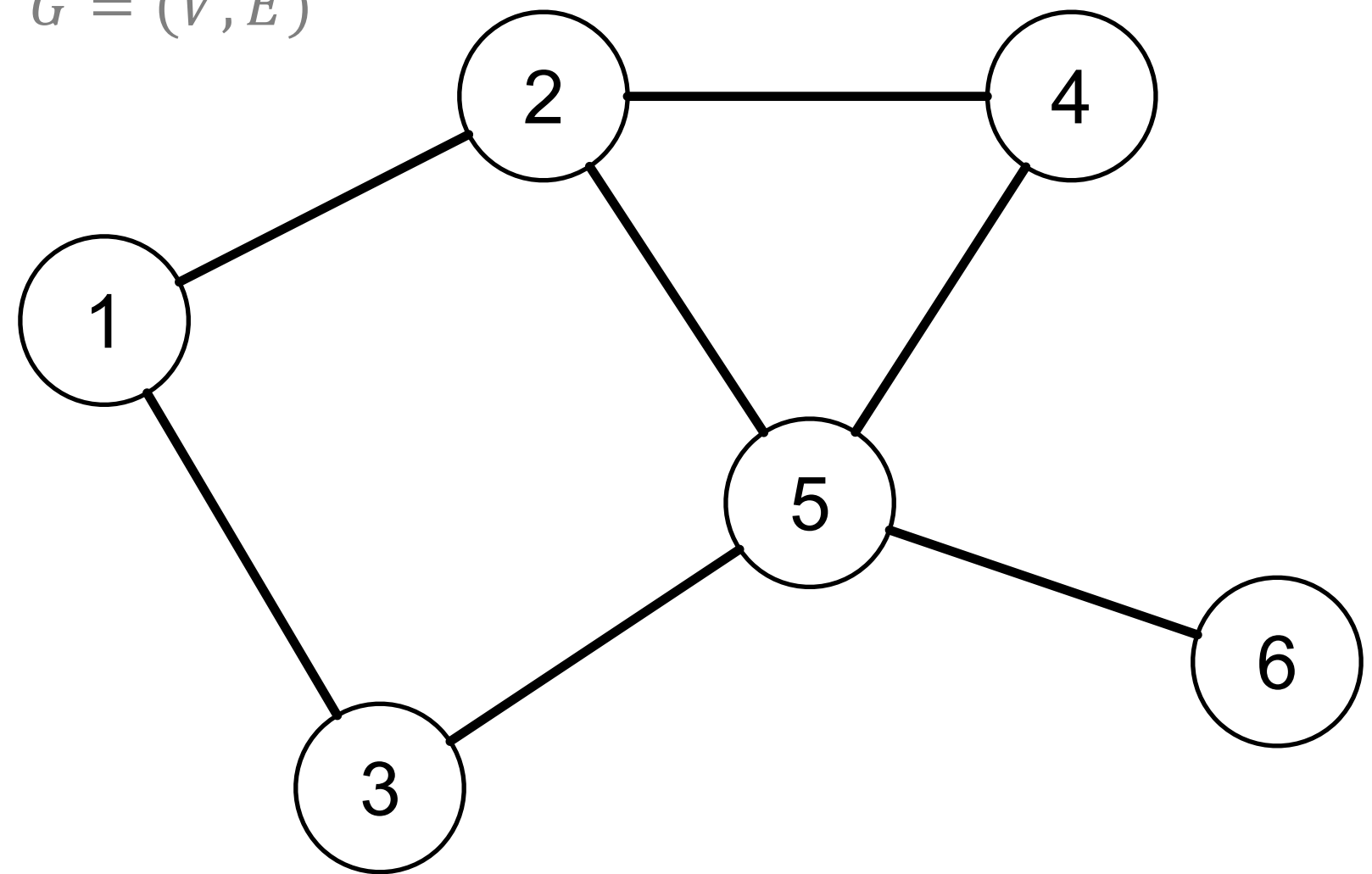
## Subgrafos

- O grafo  $H = (V', E')$  é um subgrafo gerador de  $G = (V, E)$  se  $H$  for um subgrafo de  $G$  e  $V' = V$ .

$H = (V', E')$



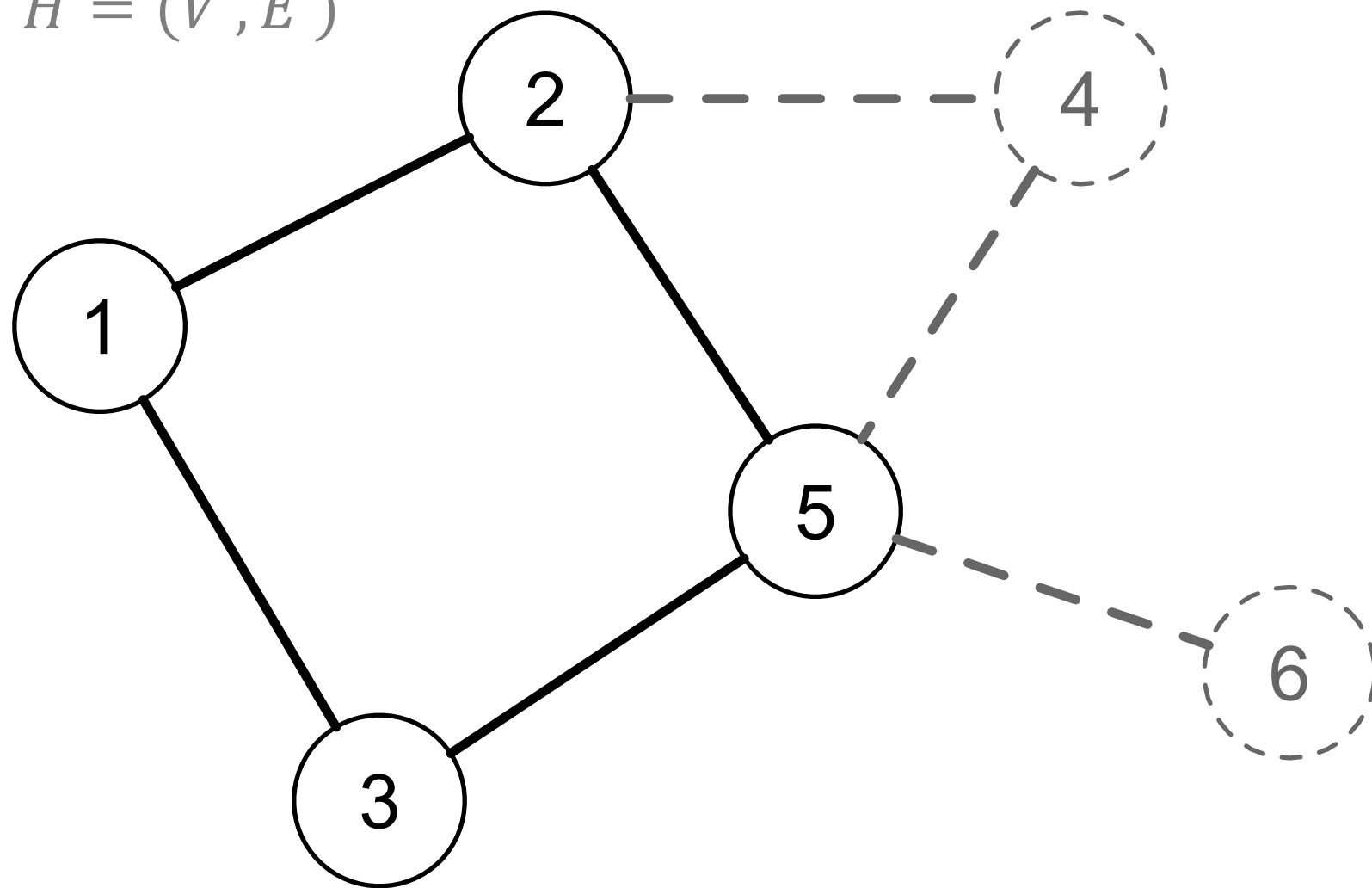
$G = (V, E)$



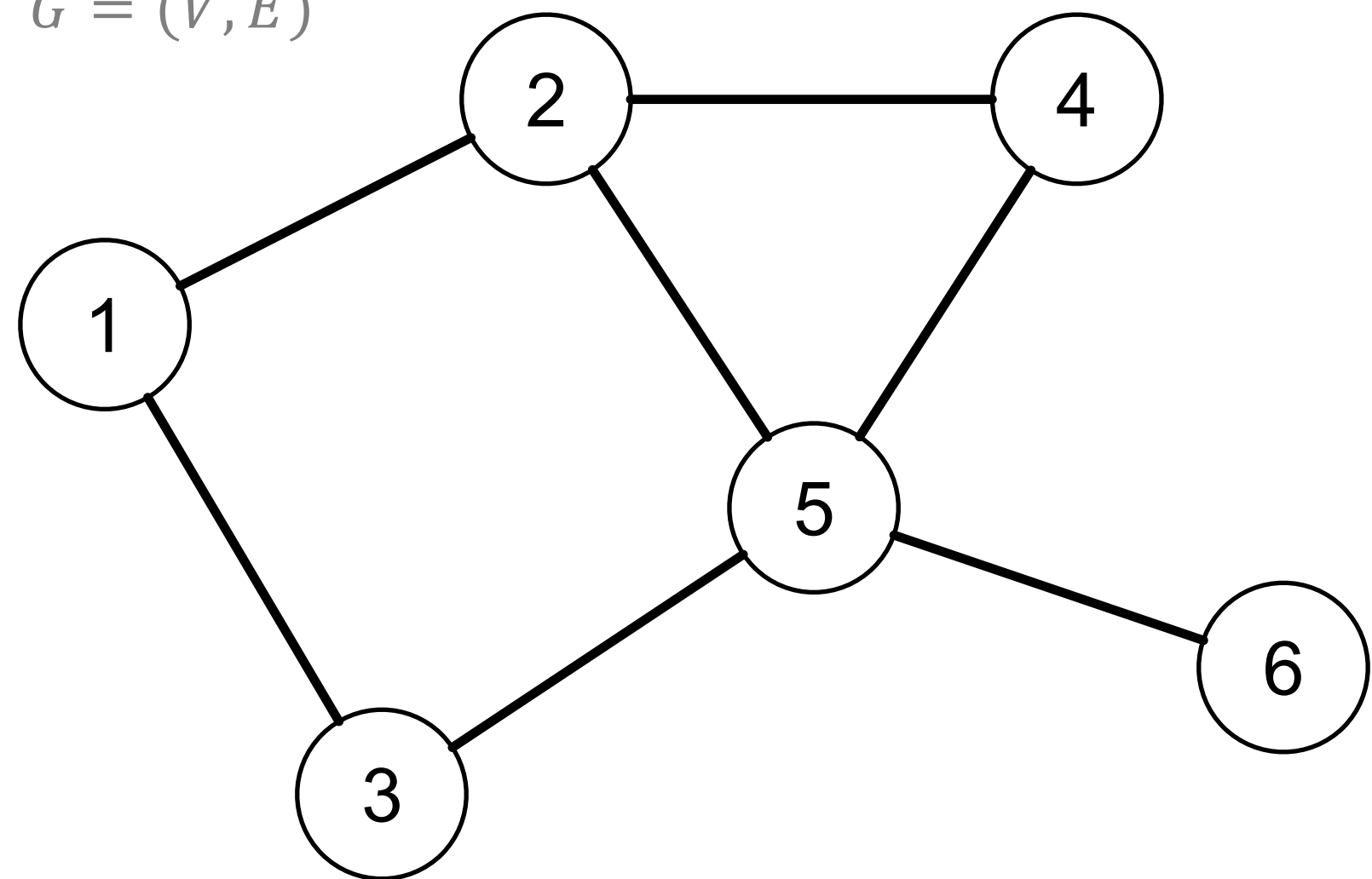
## Subgrafos

- O grafo  $H = (V', E')$  é um grafo induzido de  $G = (V, E)$  se  $E'$  for definido por todas as arestas de  $E$  adjacentes a um par de vértices em  $V'$ .

$H = (V', E')$



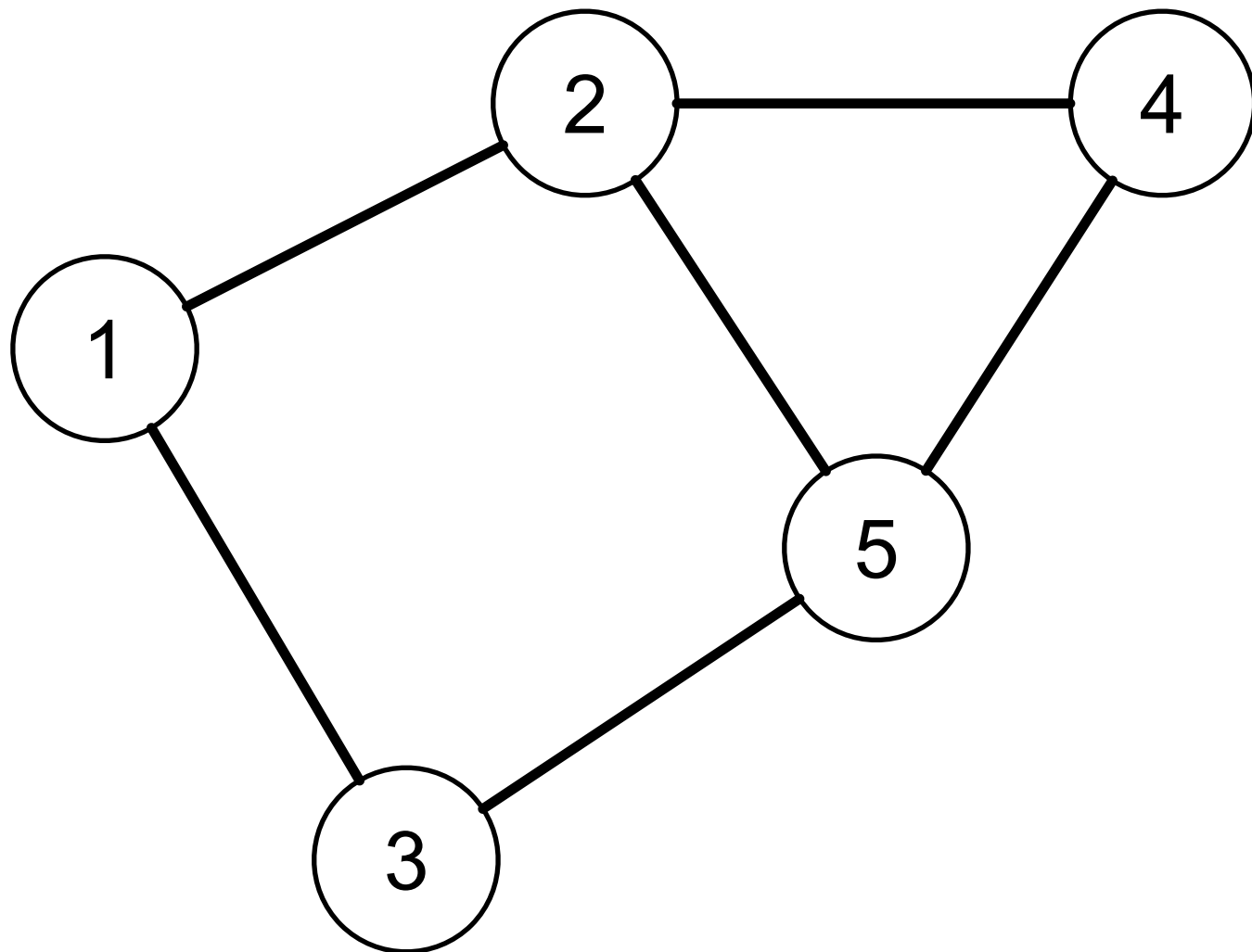
$G = (V, E)$



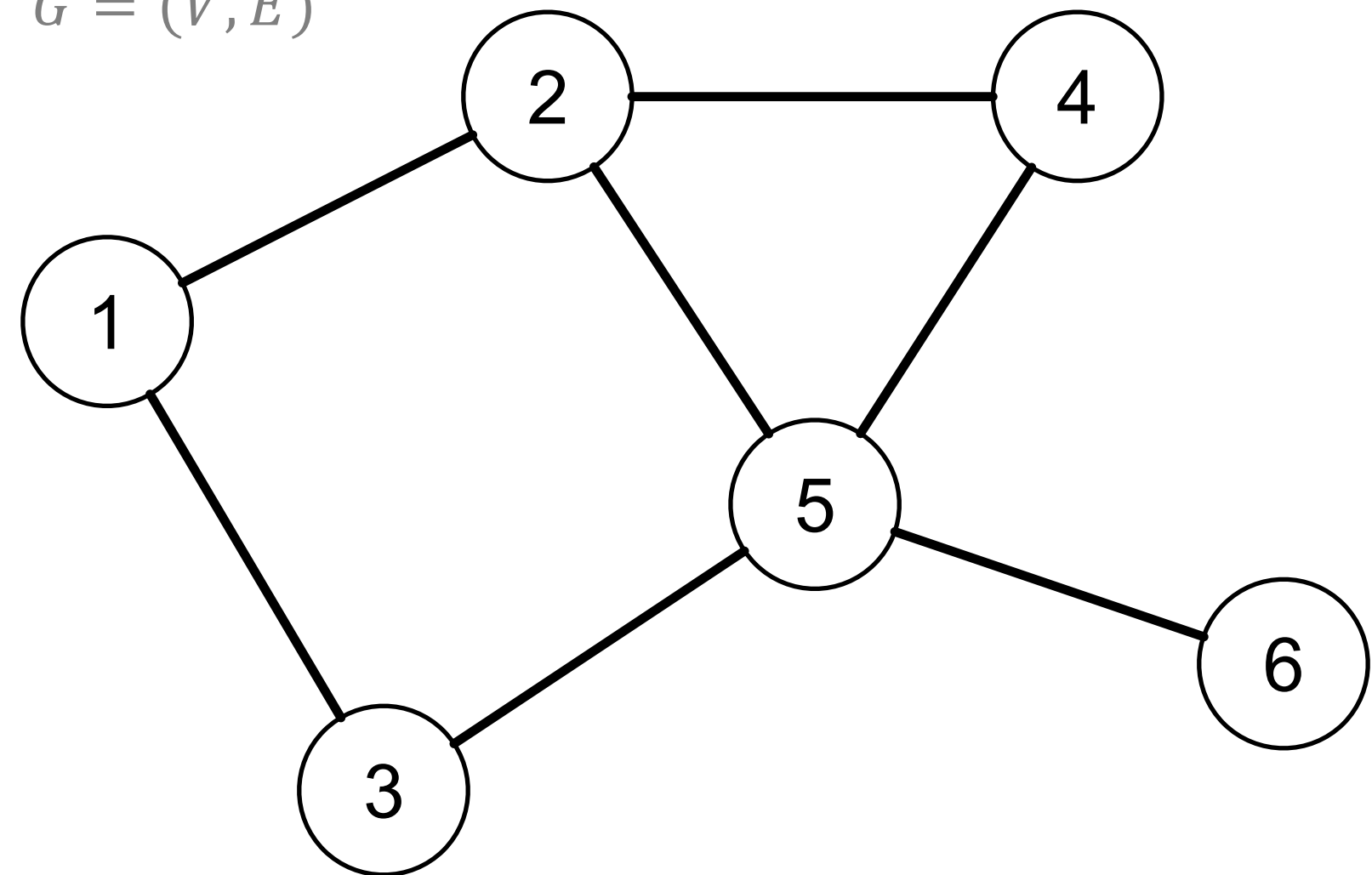
## Subgrafos

- O grafo  $H = (V', E')$  é um grafo próprio de  $G = (V, E)$  se  $H \subset G$ .

$H = (V', E')$



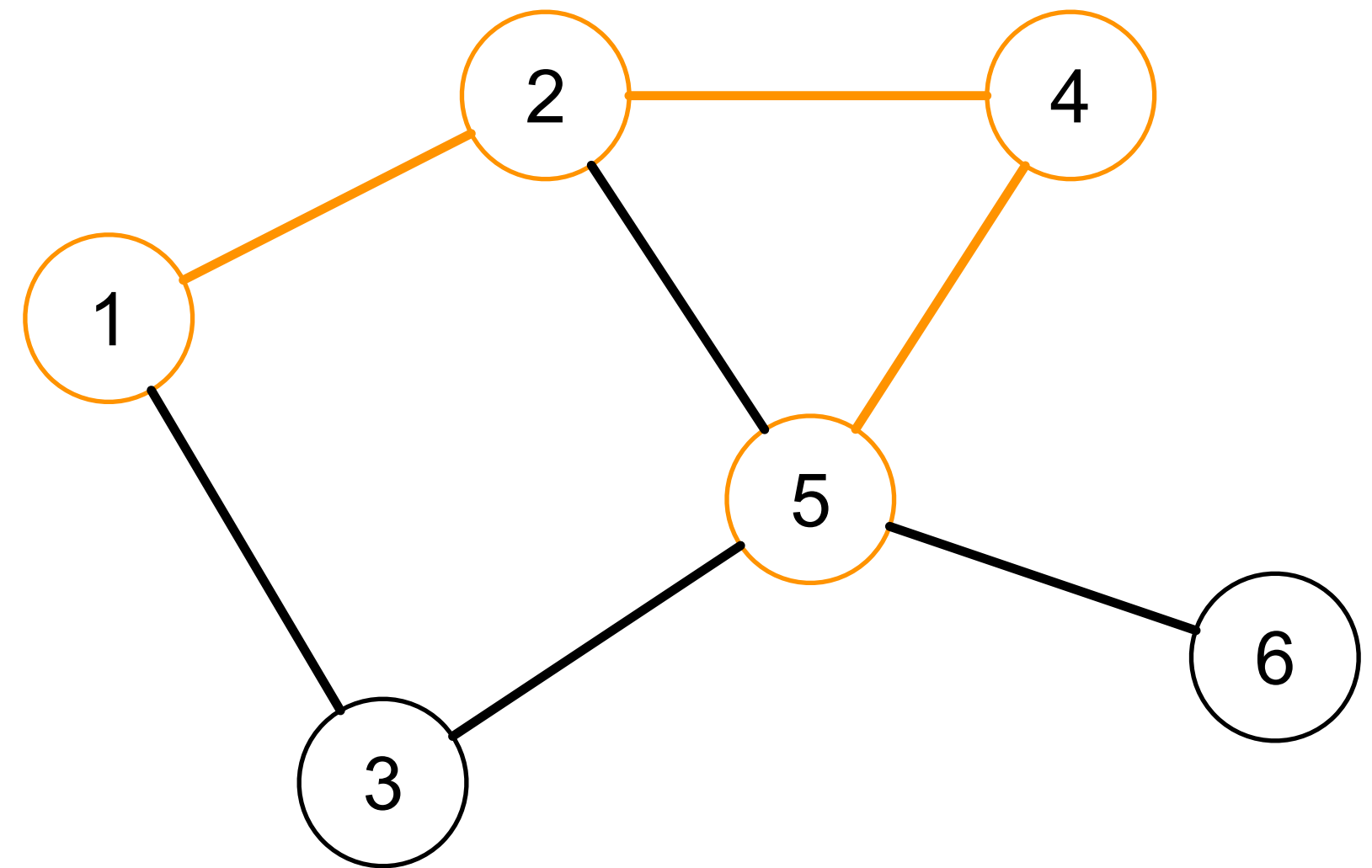
$G = (V, E)$





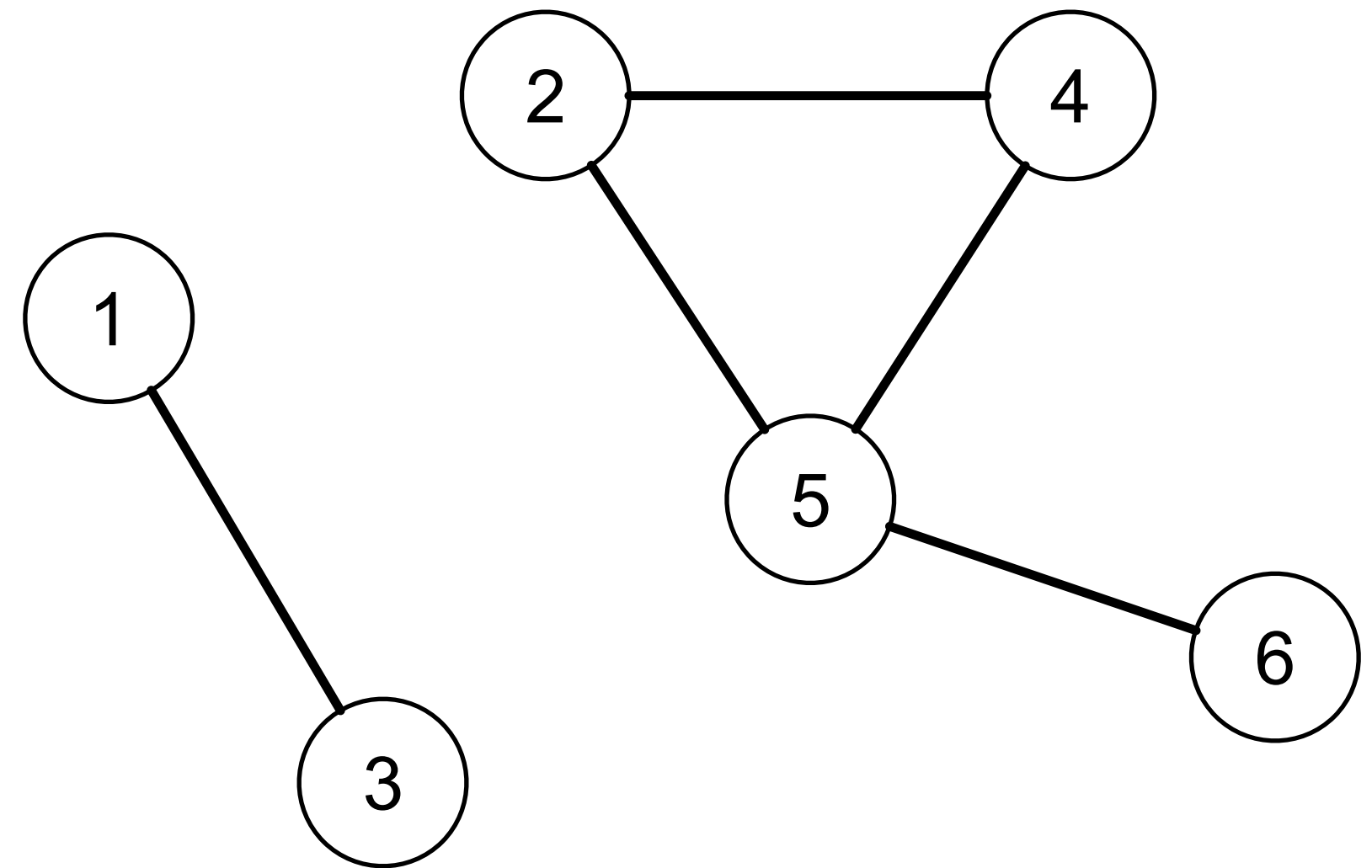
## Caminhos

- Um **caminho**  $P$  em  $G = (V, E)$  consiste em uma sequência com  $n$  vértices, finita e não vazia, tal que  $v_{i+1}$  é adjacente de  $v_i$  para  $i < n$ .
- O caminho é simple se não possuir vértices repetidos.
- O caminho é fechado se  $v_1 = v_n$ .
- O comprimento de um caminho é definido pelo número de arestas presentes nele
  - Um caminho que passa por  $n$  vértices possui  $n - 1$  arestas.



## Características

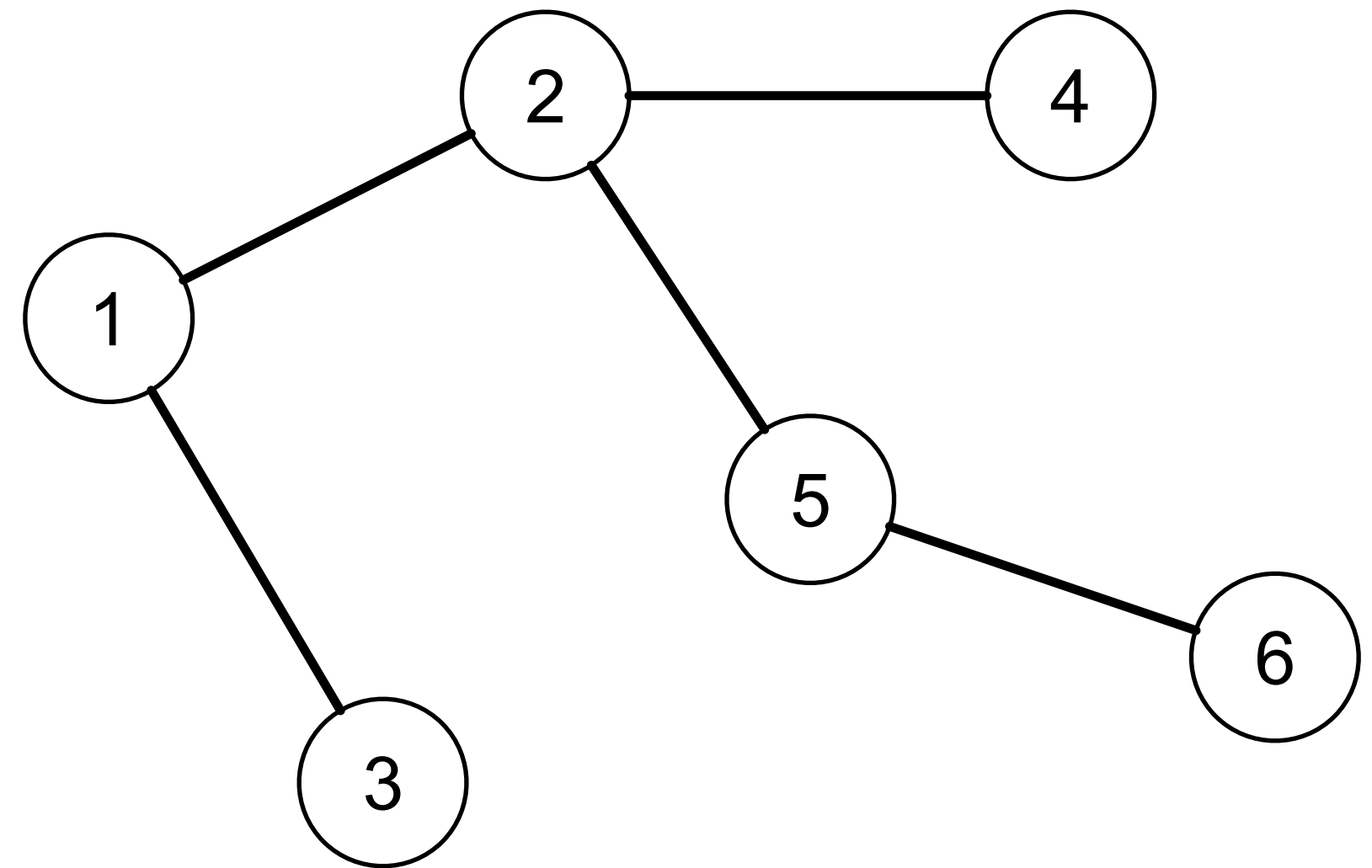
- Um grafo  $G = (V, E)$  é **conexo** se para qualquer par de vértices  $(v_i, v_j)$  existe um caminho em  $G$ .
- Quando um grafo  $G$  não é conexo podemos segmentá-lo em componentes conexos
  - Um par de vértices  $(v_i, v_j)$  está em um mesmo componente caso exista um caminho entre eles.



# Introdução à Grafos

## Características

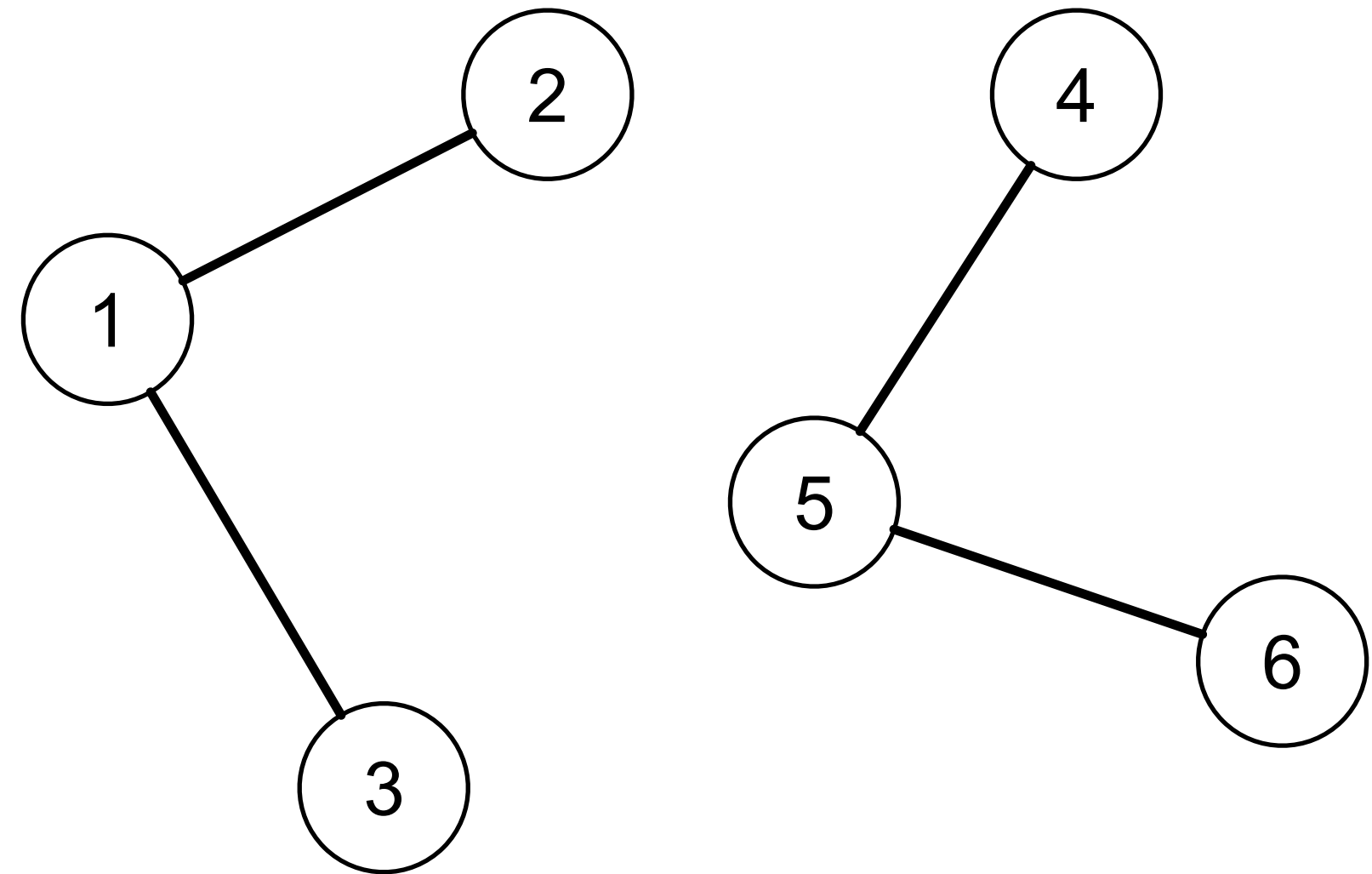
- Um grafo  $G = (V, E)$  é uma **árvore** se  $G$  for conexo e acíclico.
- Possui exatamente  $|V| - 1$  arestas.
- A remoção de qualquer aresta torna o grafo não-conexo.
- Para todo par de vértices existe um único caminho.



# Introdução à Grafos

## Características

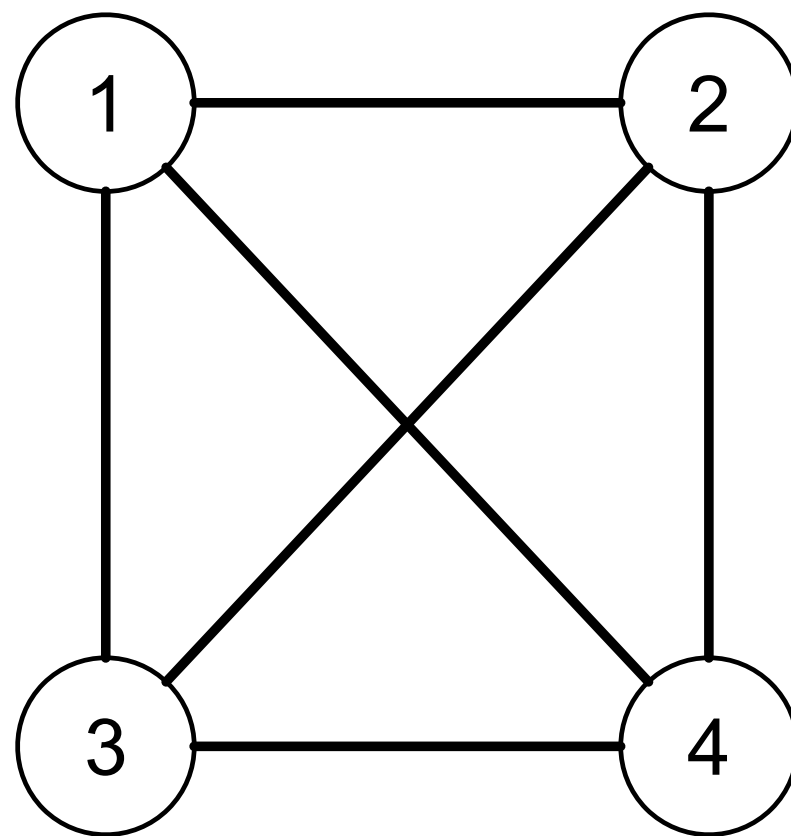
- Um grafo  $G = (V, E)$  é uma **floresta** se for um grafo acíclico.
- Observe que não é necessário que  $G$  seja conexo.



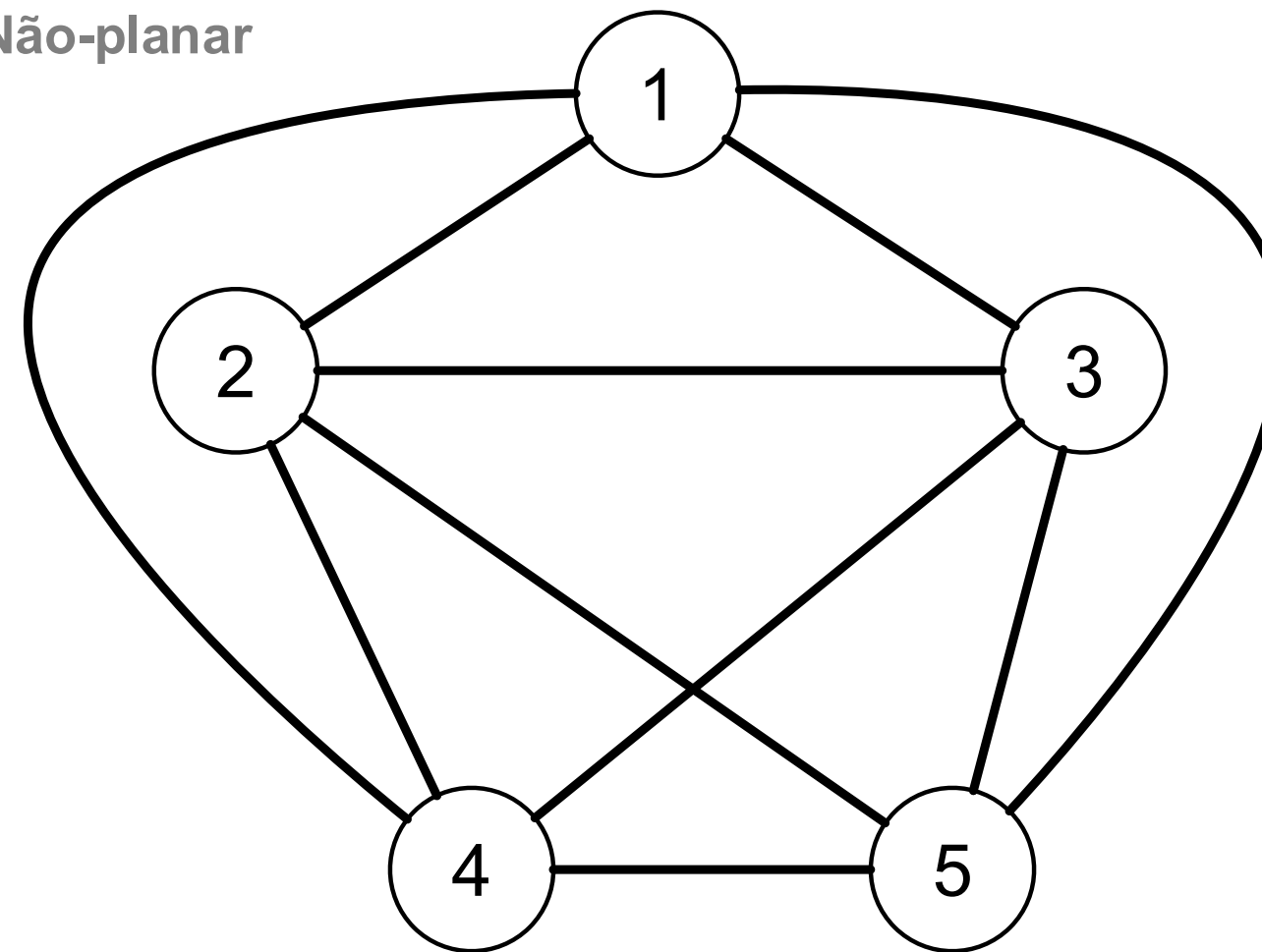
## Características

- Um grafo  $G = (V, E)$  é **planar** se puder ser representado graficamente em um plano de tal forma que não haja cruzamento de suas arestas.

Planar



Não-planar

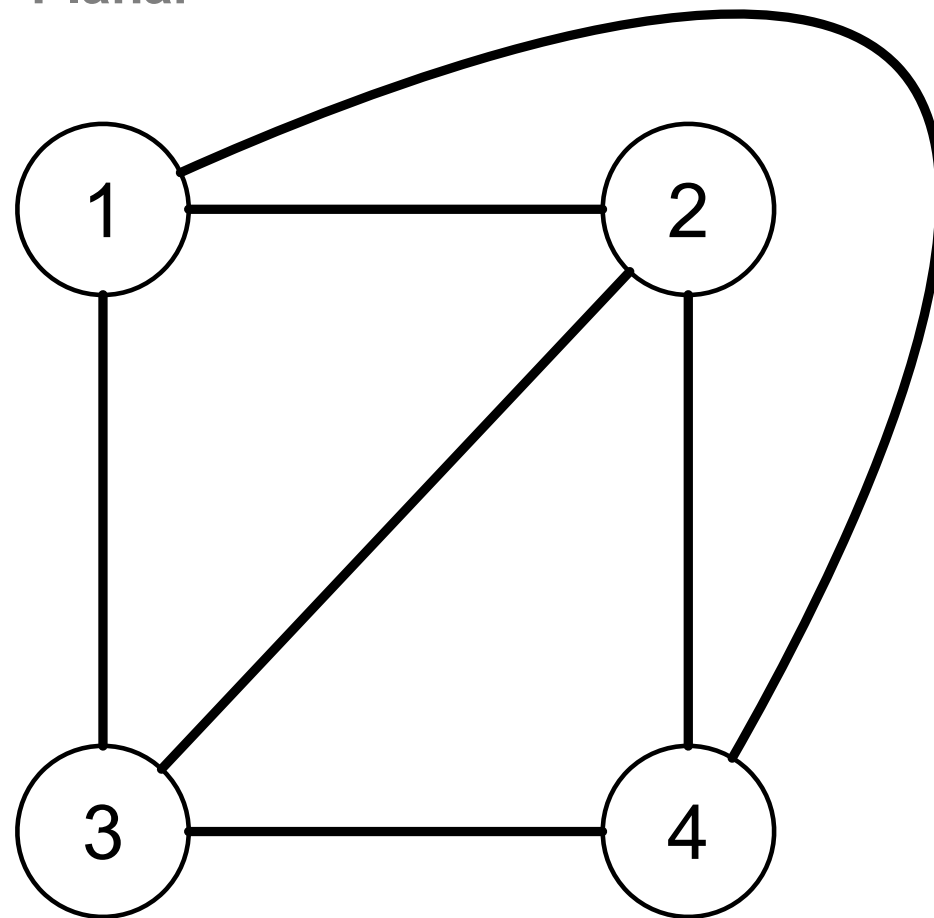




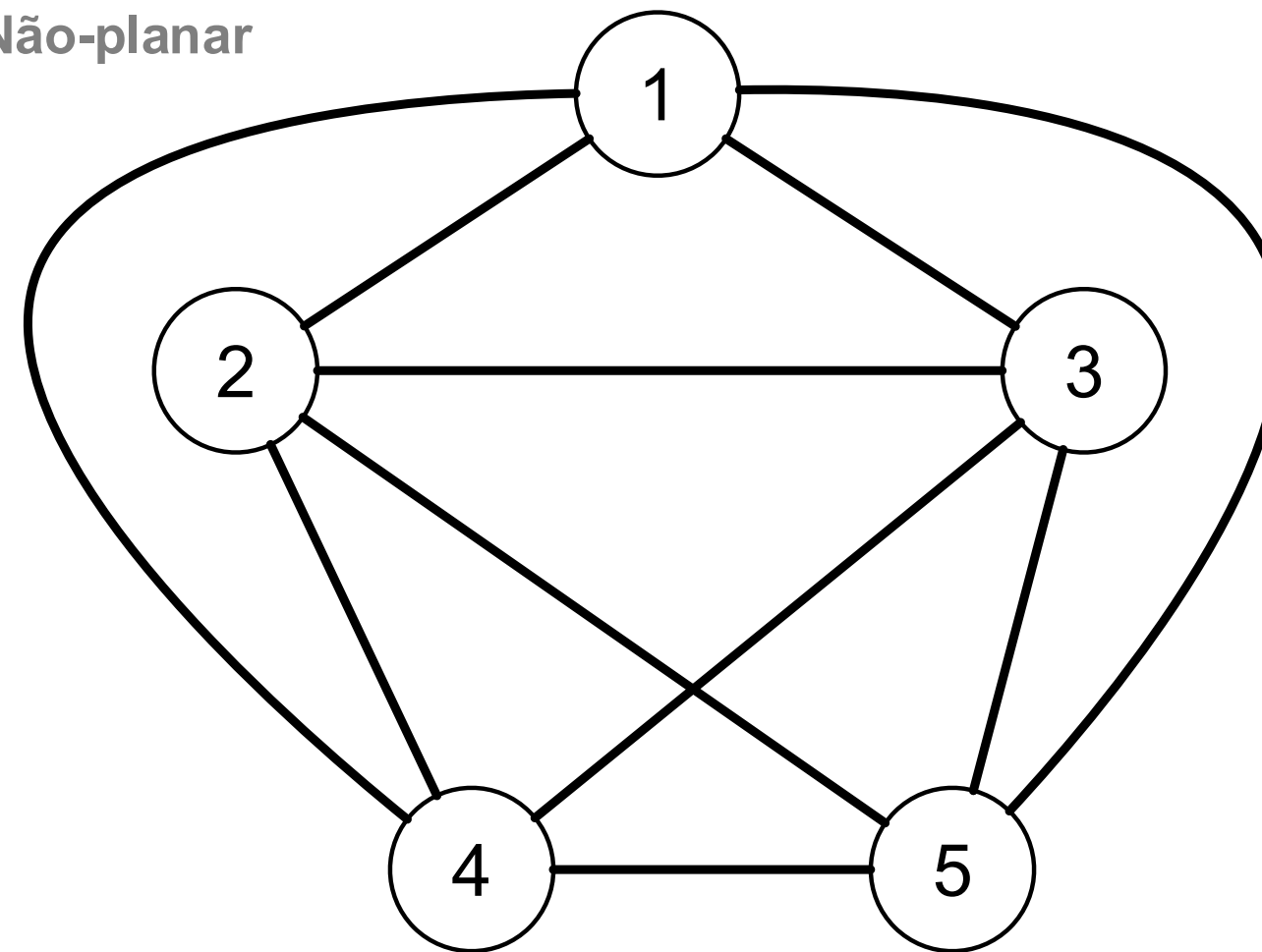
## Características

- Um grafo  $G = (V, E)$  é **planar** se puder ser representado graficamente em um plano de tal forma que não haja cruzamento de suas arestas.

Planar

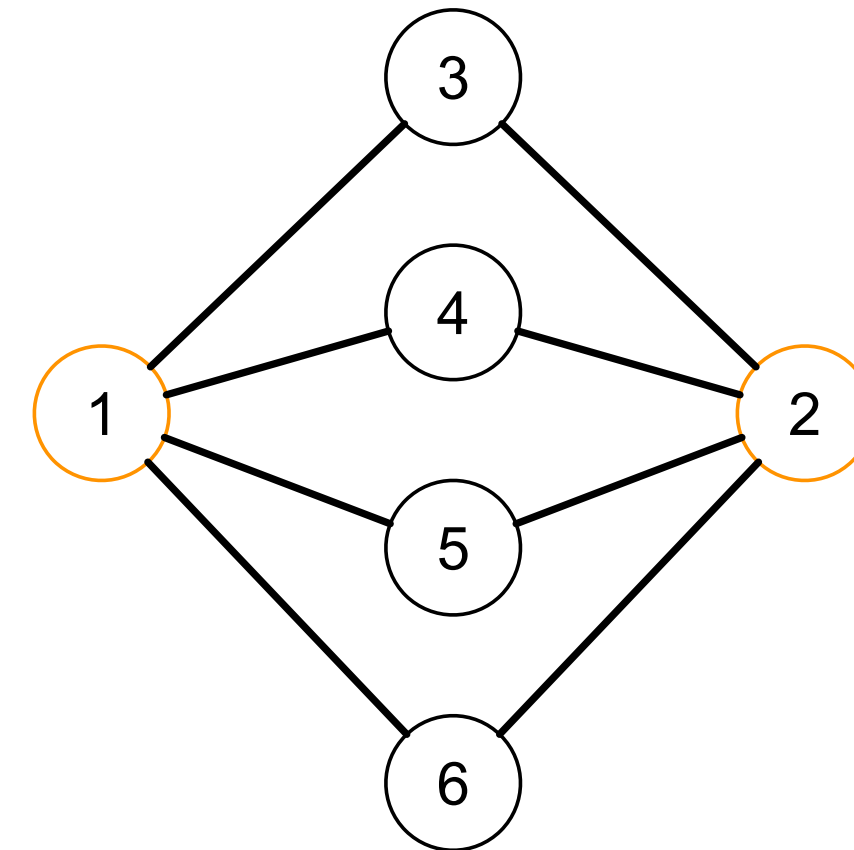
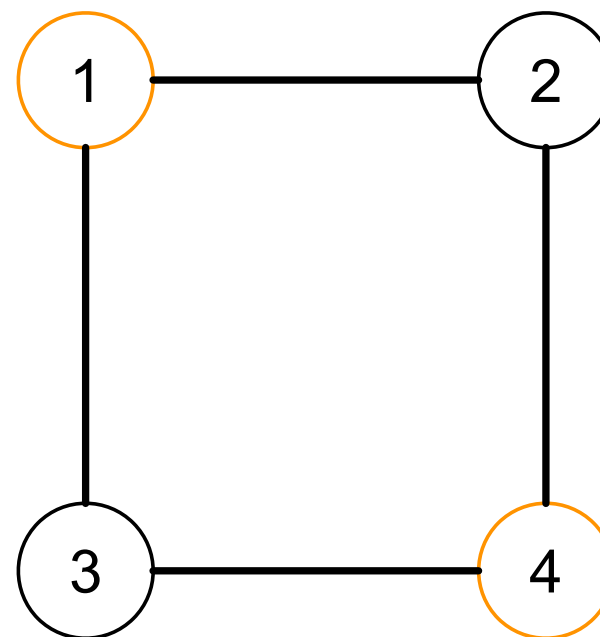
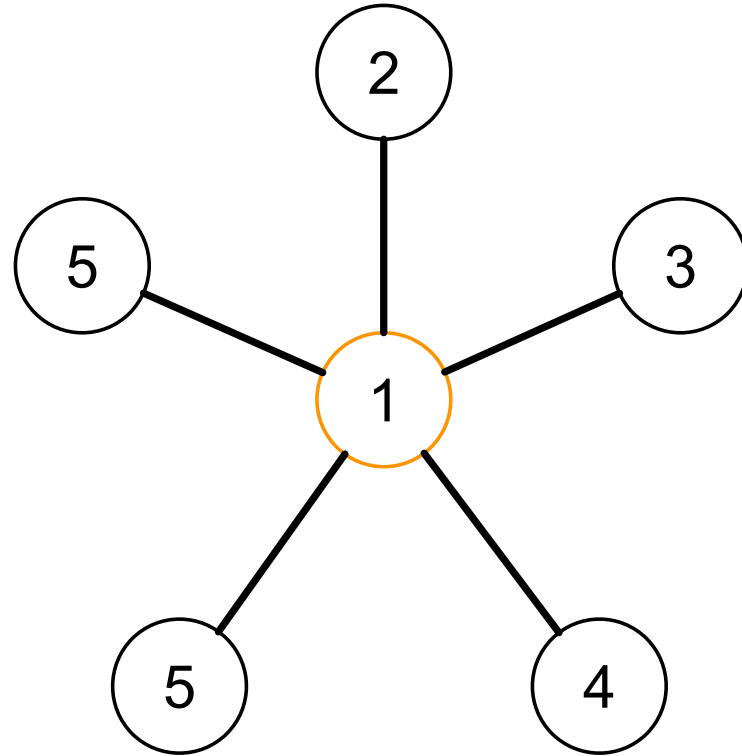


Não-planar

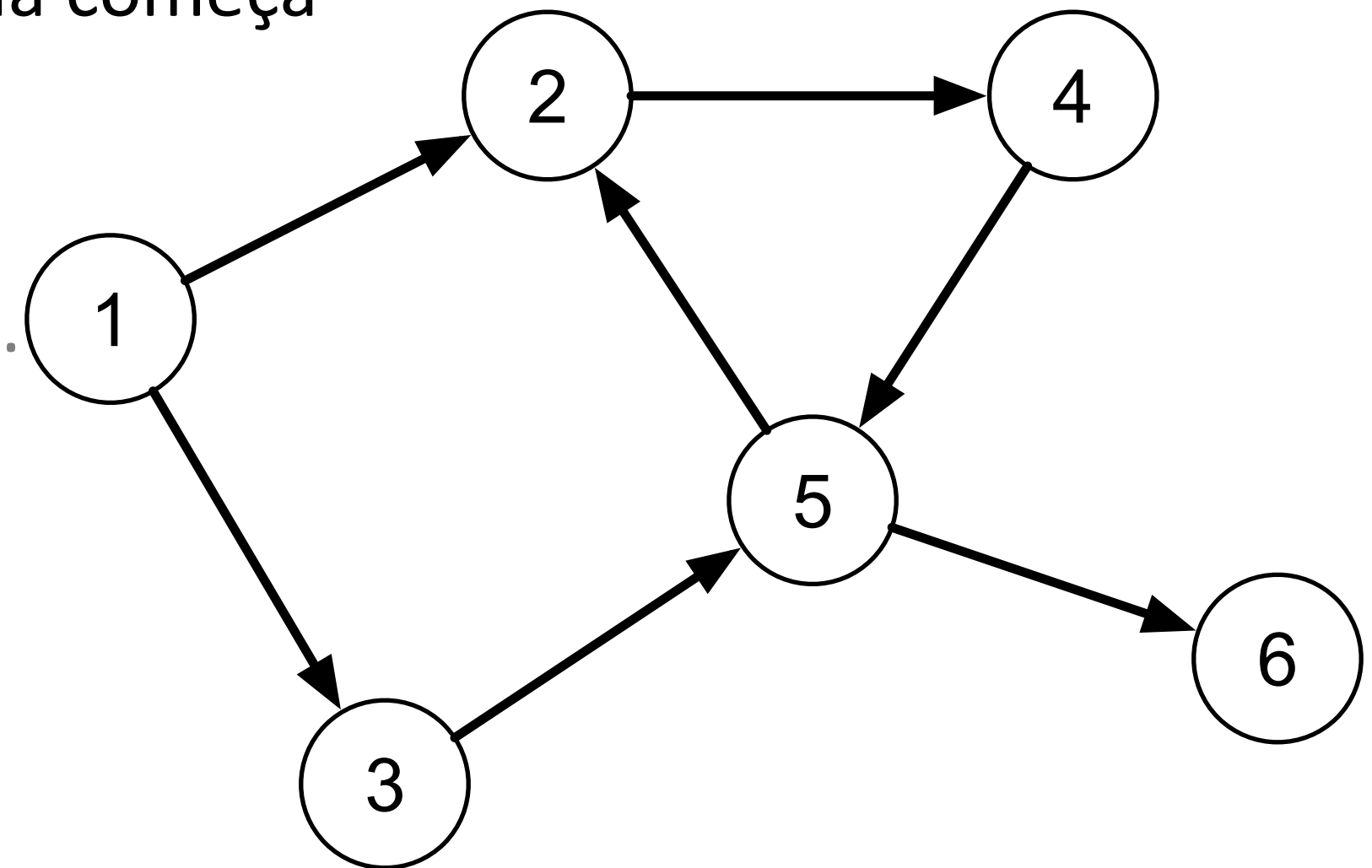


## Características

- Um grafo  $G = (V, E)$  é **bipartido** se os vértices puderem ser divididos em dois conjuntos  $V_1$  e  $V_2$  de forma que toda aresta  $e_k$  é incidente em  $(v_i, v_j)$  tal que  $v_i \in V_1$  e  $v_j \in V_2$ .



- Chamamos de **grafo orientado** (ou dirigido) quando as arestas possuem um sentido.
- Dada uma aresta  $e = (v_i, v_j)$  dizemos que ela começa em  $v_i$  e termina em  $v_j$ 
  - O vértice  $v_j$  é adjacente de  $v_i$ , no entanto  $v_i$  não é necessariamente adjacente de  $v_j$ .
- A representação do grafo deverá considerar a direção das arestas:
  - $E = \{(1,2), (1,3), (2,4), (5, 2), (3,5), (4,5), (5,6)\}$

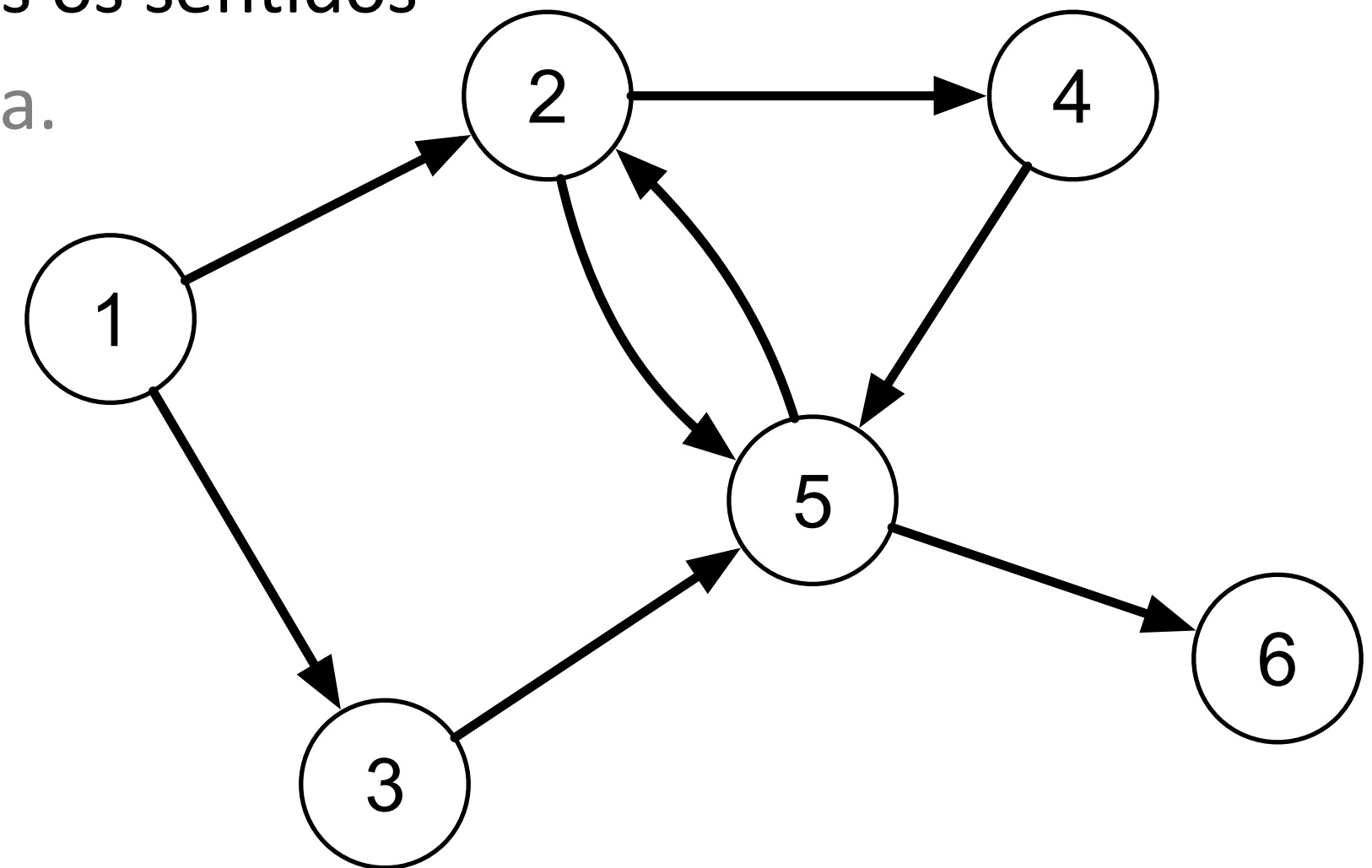




# Introdução à Grafos

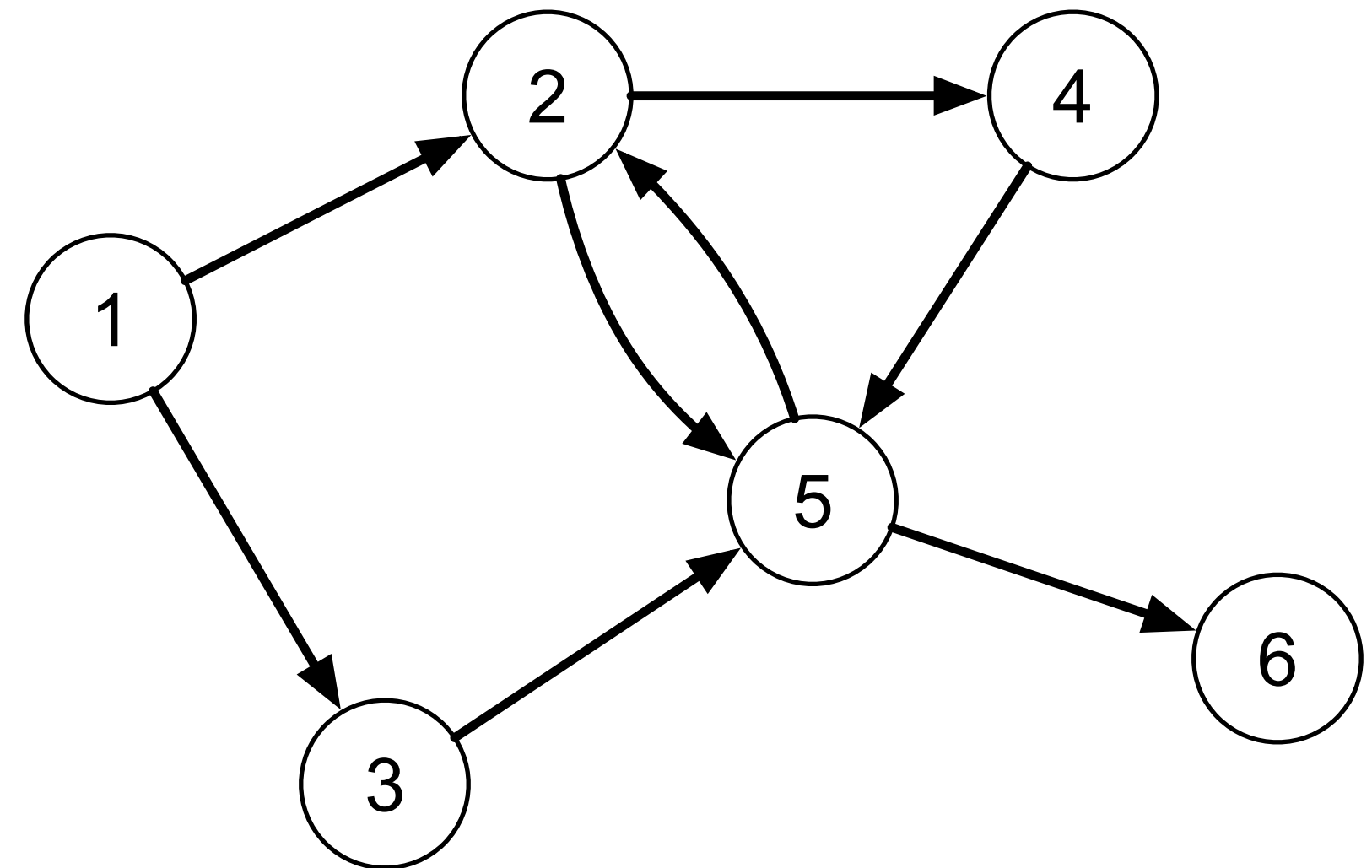
## Grafos orientados

- Observe que o grafo pode apresentar mais de uma aresta entre dois vértices.
- Dessa forma podem existir arestas em ambos os sentidos
  - Tornando a relação de adjacência simétrica.
  - Essas arestas são antiparalelas.
- Podemos representar essas arestas como:
  - $E = \{(1,2), (1,3), (2,4), (2,5), (5,2)$   
 $(3,5), (4,5), (5,6)\}$



- O **grau de saída**  $g_s(v_i)$  é definido pelo número de arestas que saem de  $v_i$ .
- Analogamente o **grau de entrada**  $g_e(v_i)$  é definido pelo número de arestas que chegam em  $v_i$ .
- Também podemos afirmar que:

$$\sum_{i=1}^{|V|} g_e(v_i) = \sum_{i=1}^{|V|} g_s(v_i) = |E|$$



# Introdução à Grafos

## Grafos orientados

- O vértice  $v_i$  é uma fonte se:

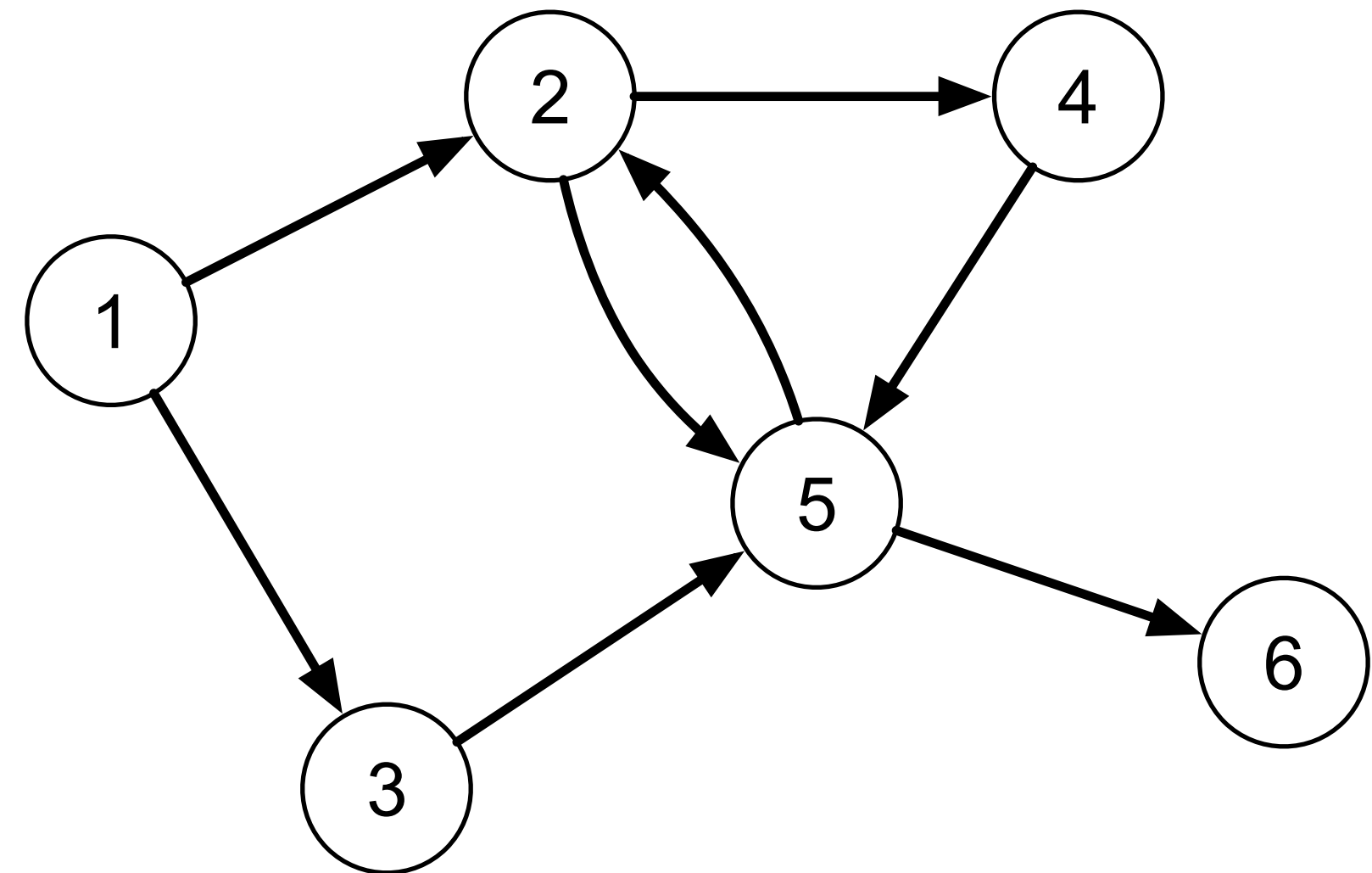
$$g_e(v_i) = 0$$

- O vértice  $v_i$  é um sorvedouro se:

$$g_s(v_i) = 0$$

- O vértice  $v_i$  é isolado se:

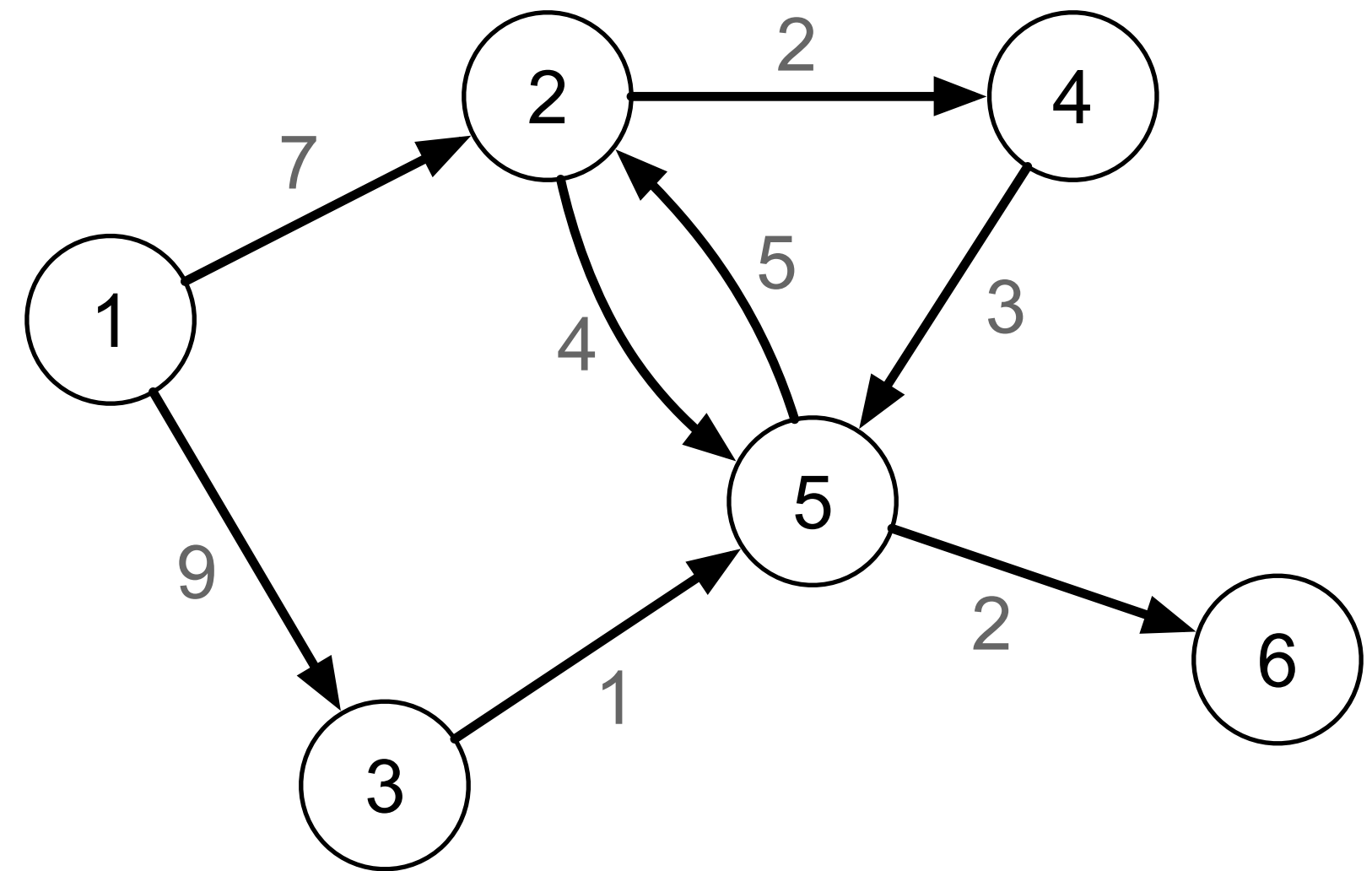
$$g_e(v_i) = 0 \text{ e } g_s(v_i) = 0$$



# Introdução à Grafos

## Grafos ponderados

- Um grafo (orientado ou não) é ponderado se cada aresta estiver associada à um valor.
- O nome dessa propriedade varia de acordo com a natureza do problema
  - Custo.
  - Peso.
  - Capacidade.
  - Etc.



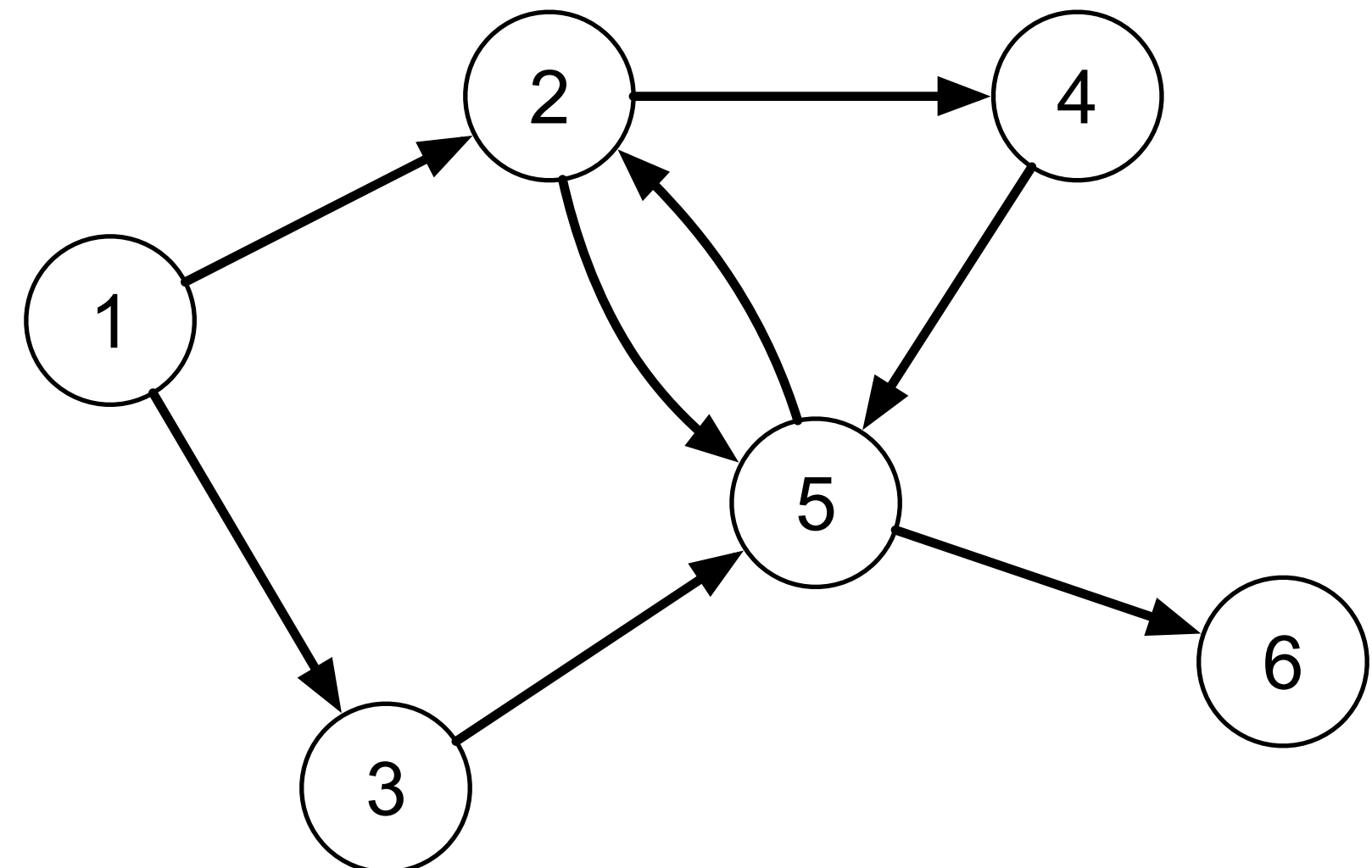
# **Estruturas de dados para representar grafos**



- Algoritmos sobre grafos dependem de uma estrutura de dados em memória para representar a estrutura do mesmo.
- Existem duas formas comuns de implementar essa representação:
  - Matriz de adjacência.
  - Lista de adjacência.
- A escolha da representação depende do algoritmo a ser implementado
  - Dada a natureza das operações o desempenho do algoritmo pode variar de acordo com essa escolha.

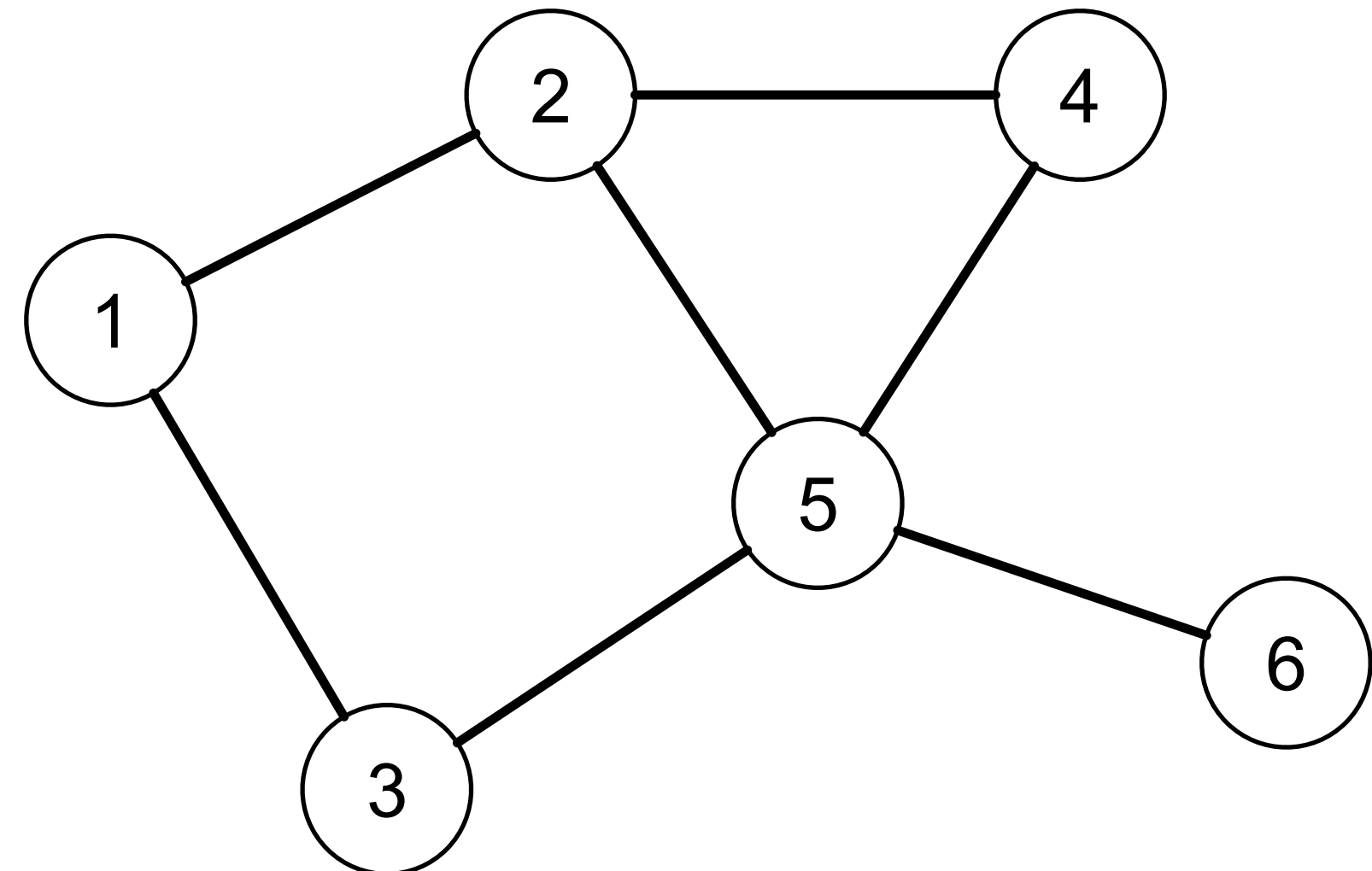
- A estrutura de dados **matriz de adjacência** consiste em um matriz quadrada  $A$  de ordem  $|V|$  cujas linhas e colunas são indexadas pelos vértices de  $V$ .

$$\begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$



- A estrutura de dados **matriz de adjacência** consiste em um matriz quadrada  $A$  de ordem  $|V|$  cujas linhas e colunas são indexadas pelos vértices de  $V$ .

$$\begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$





- A complexidade de acessar (ou verificar) uma aresta é  $\Theta(1)$ .
- Uma matriz de adjacência exige um espaço  $\Theta(|V|^2)$ .
- Se o grafo for não-dirigido a matriz de  $A$  é igual à sua transposta  $A^T$ .
  - Dependendo do problema pode-se armazenar somente a diagonal e os elementos superiores, reduzindo pela metade o espaço necessário.
- A matriz pode ser implementada através de bits de forma que o espaço seja reduzido.

- **Exercício 1:** implemente uma classe para representar um grafo utilizando matriz de adjacências.

```
typedef int vertex;

class GraphMatrix {
    GraphMatrix(int numVertices);

    ~GraphMatrix();

    bool hasEdge(vertex v1, vertex v2);

    void addEdge(vertex v1, vertex v2);

    void removeEdge(vertex v1, vertex v2);

    void print();

    void printMatrix();
};
```

- **Exercício 1:** implemente uma classe para representar um grafo utilizando matriz de adjacências.

```
GraphMatrix(int numVertices)    // The number of vertices will be fixed
: m_numVertices(numVertices)
, m_numEdges(0)
, m_edges(nullptr) {
m_edges = new bool*[m_numVertices];    // Allocate the edge matrix
for (vertex i=0; i < m_numVertices; i++) {
    m_edges[i] = new bool[m_numVertices];
    for (vertex j=0; j < m_numVertices; j++) {
        m_edges[i][j] = false;    // Initialize
    }
}
}

~GraphMatrix() {
    for (int i=0; i < m_numVertices; i++) {
        delete[] m_edges[i];
    }
    delete[] m_edges;
}
```

- **Exercício 1:** implemente uma classe para representar um grafo utilizando matriz de adjacências.

```
bool hasEdge(vertex v1, vertex v2) {  
    return m_edges[v1][v2];  
}  
  
void addEdge(vertex v1, vertex v2) {  
    if (!hasEdge(v1, v2)) {  
        m_edges[v1][v2] = true;  
        m_numEdges++;  
    }  
}  
  
void removeEdge(vertex v1, vertex v2) {  
    if (hasEdge(v1, v2)) {  
        m_edges[v1][v2] = false;  
        m_numEdges--;  
    }  
}
```

- **Exercício 1:** implemente uma classe para representar um grafo utilizando matriz de adjacências.

```
void print() {  
    for (vertex i=0; i < m_numVertices; ++i) {  
        for (vertex j=0; j < m_numVertices; ++j) {  
            if (hasEdge(i, j)) {  
                printf("(%d,%d) ", i, j);  
            }  
        }  
        printf("\n");  
    }  
}
```

```
void printMatrix() {  
    for (vertex i=0; i < m_numVertices; ++i) {  
        for (vertex j=0; j < m_numVertices; ++j) {  
            printf("%d ", hasEdge(i, j));  
        }  
        printf("\n");  
    }  
}
```



- **Exercício 1:** implemente uma classe para representar um grafo utilizando matriz de adjacências.

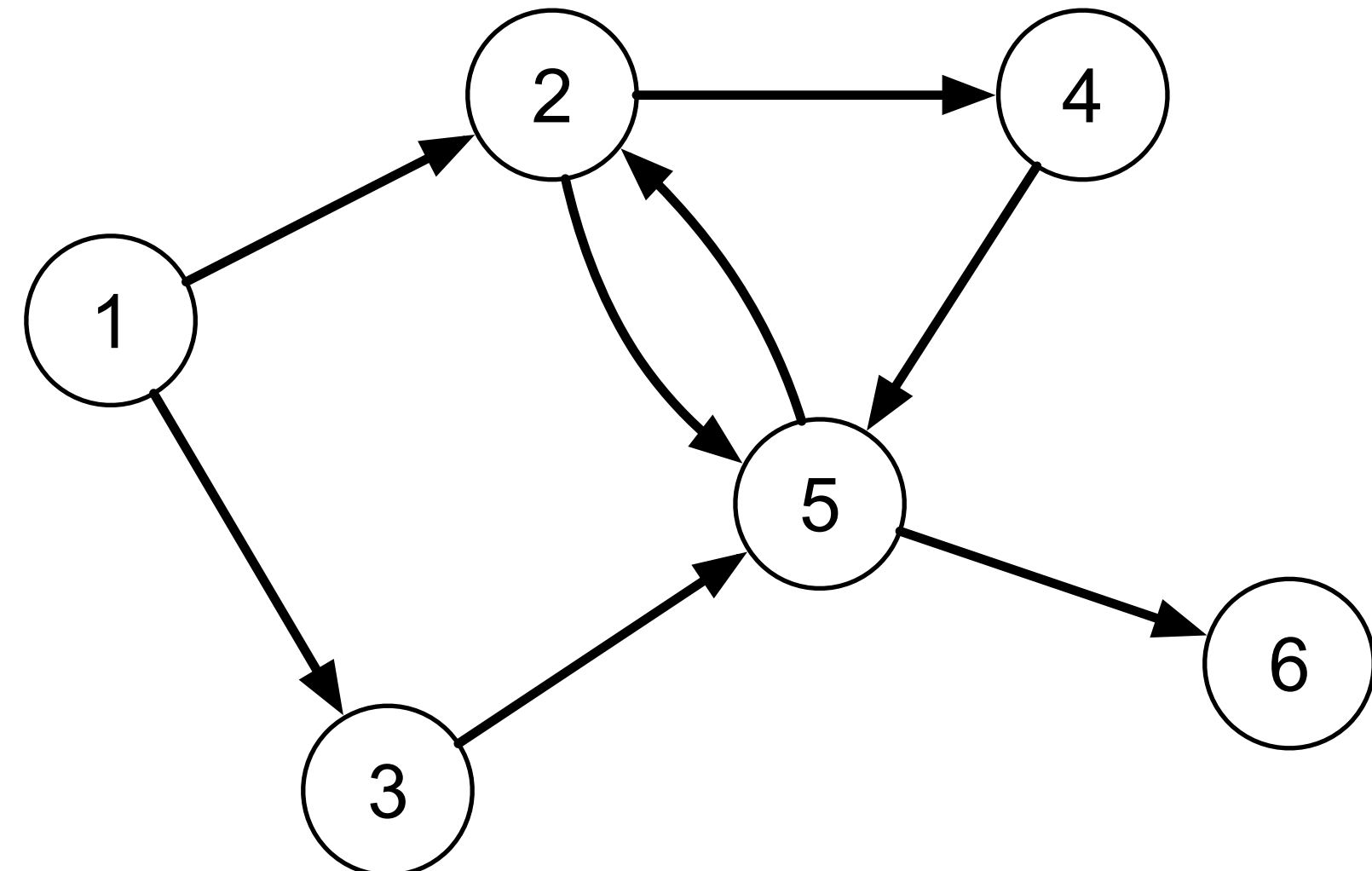
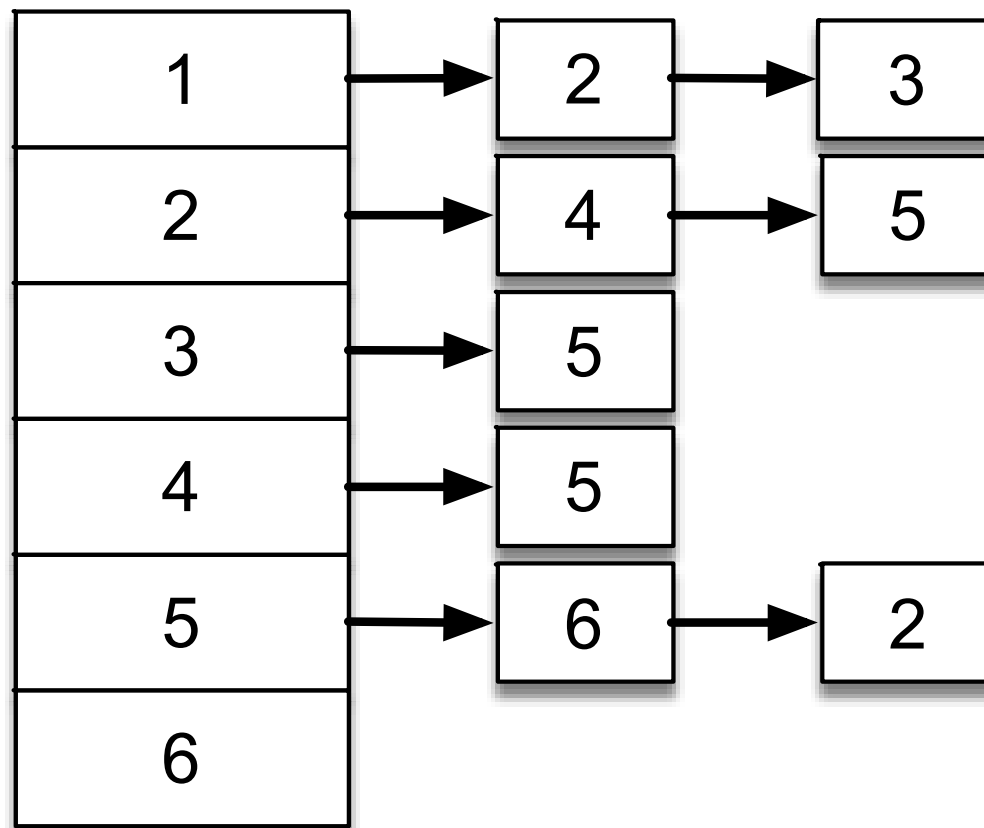
```
GraphMatrix g1(6);  
g1.addEdge(0, 1);  
g1.addEdge(0, 2);  
g1.addEdge(1, 3);  
g1.addEdge(1, 4);  
g1.addEdge(2, 4);  
g1.addEdge(3, 4);  
g1.addEdge(4, 5);  
g1.addEdge(4, 1);  
g1.print();  
g1.printMatrix();
```

**Output:**

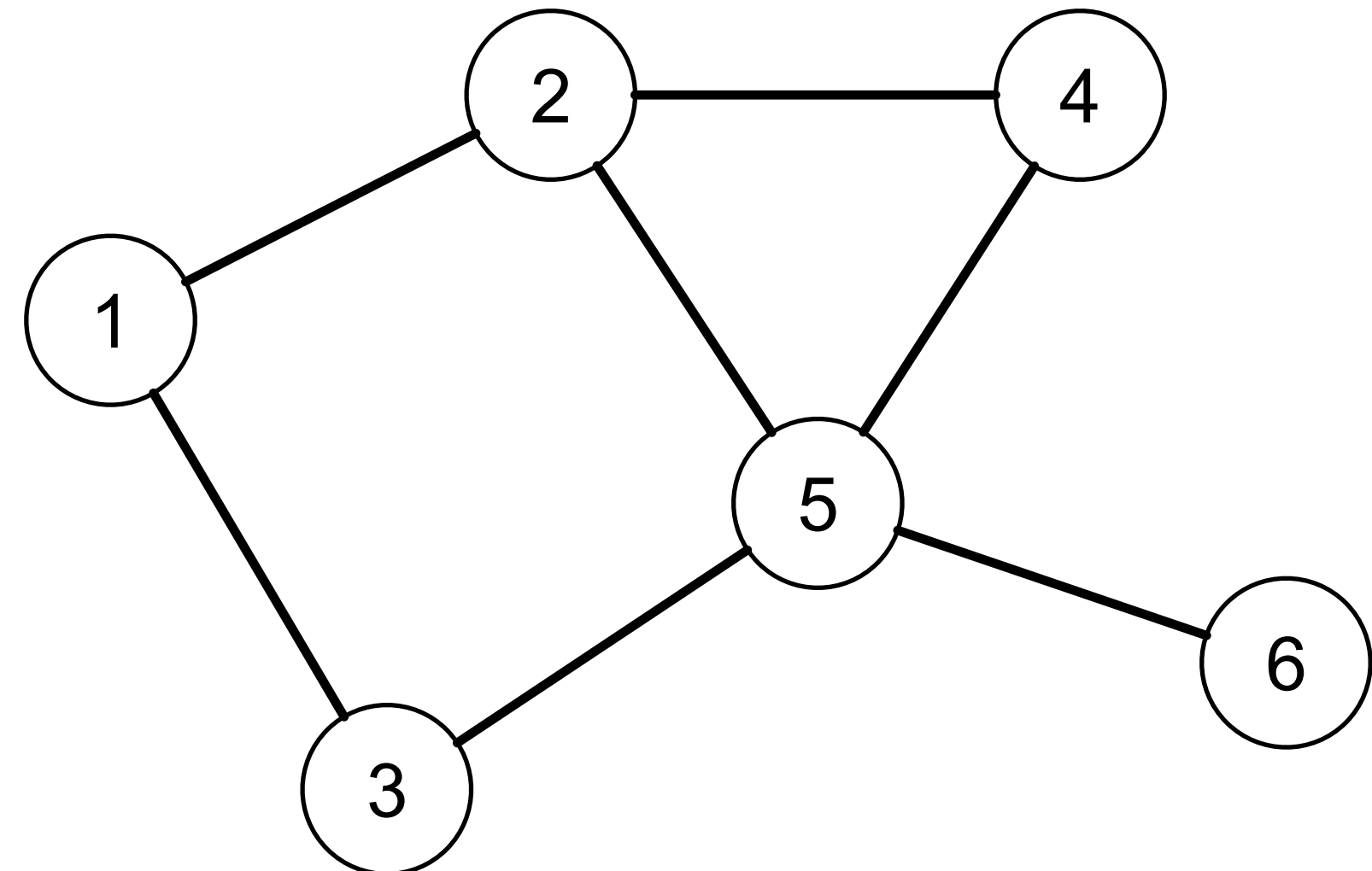
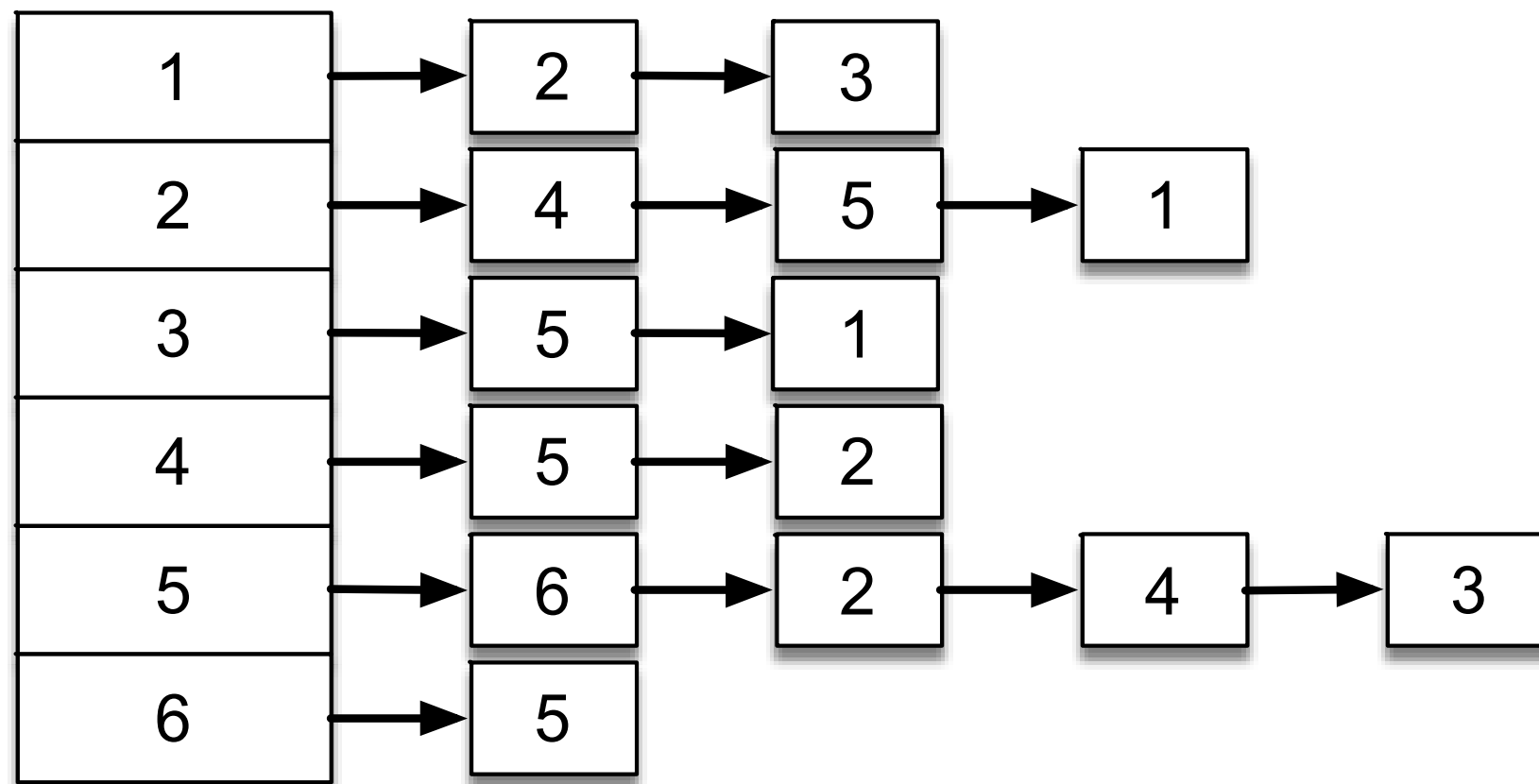
```
(0, 1) (0, 2)  
(1, 3) (1, 4)  
(2, 4)  
(3, 4)  
(4, 1) (4, 5)
```

```
0 1 1 0 0 0  
0 0 0 1 1 0  
0 0 0 0 1 0  
0 0 0 0 1 0  
0 1 0 0 0 1  
0 0 0 0 0 0
```

- A estrutura de dados **lista de adjacência** consiste em uma sequência de vértices contendo na estrutura de cada um ponteiro para uma lista encadeada com elementos representando as arestas adjacentes ao vértice.



- A estrutura de dados **lista de adjacência** consiste em uma sequência de vértices contendo na estrutura de cada um ponteiro para uma lista encadeada com elementos representando as arestas adjacentes ao vértice.





- A complexidade de acessar o conjunto de arestas de um vértice é  $\theta(1)$ 
  - No entanto a complexidade de encontrar uma aresta específica é  $\theta(|V|)$  no pior caso.
- Uma lista de adjacência exige um espaço  $\theta(|V| + |E|)$ .
- As estruturas de dados do vértice e da aresta podem ser estendidas para armazenar informações específicas do problema.

- **Exercício 2:** implemente uma classe para representar um grafo utilizando lista de adjacências.

```
class GraphAdjList {
public:
    GraphAdjList(int numVertices);

    ~GraphAdjList();

    void addEdge(vertex v1, vertex v2);

    void removeEdge(vertex v1, vertex v2);

    void print();

private:
    int m_numVertices;
    int m_numEdges;
    EdgeNode ** m_edges;
};
```

- **Exercício 2:** implemente uma classe para representar um grafo utilizando lista de adjacências.

```
class EdgeNode {
public:
    EdgeNode(vertex otherVertex, EdgeNode * next)
        : m_otherVertex(otherVertex)
        , m_next(next) {}

    vertex otherVertex() {
        return m_otherVertex;
    }

    EdgeNode * next() {
        return m_next;
    }

    void setNext(EdgeNode * next) {
        m_next = next;
    }

private:
    int m_otherVertex;
    EdgeNode * m_next;
};
```

- **Exercício 2:** implemente uma classe para representar um grafo utilizando lista de adjacências.

```
GraphAdjList(int numVertices)
: m_numVertices(numVertices)
, m_numEdges(0)
, m_edges(nullptr) {
    m_edges = new EdgeNode*[numVertices];
    for (vertex i=0; i < numVertices; i++) {
        m_edges[i] = nullptr;
    }
}
```

```
~GraphAdjList() {
    for (vertex i=0; i < m_numVertices; i++) {
        EdgeNode * edge = m_edges[i];
        while (edge) {
            EdgeNode * next = edge->next();
            delete edge;
            edge = next;
        }
    }
    delete[] m_edges;
}
```

- **Exercício 2:** implemente uma classe para representar um grafo utilizando lista de adjacências.

```
void addEdge(vertex v1, vertex v2) {  
    EdgeNode * edge = m_edges[v1];  
    while (edge) {  
        if (edge->otherVertex() == v2) { // Check if exists  
            return;  
        }  
        edge = edge->next();  
    }  
    m_edges[v1] = new EdgeNode(v2, m_edges[v1]); // Create the new edge  
    m_numEdges++;  
}
```



- **Exercício 2:** implemente uma classe para representar um grafo utilizando lista de adjacências.

```
void removeEdge(vertex v1, vertex v2) {  
    EdgeNode * edge = m_edges[v1];  
    EdgeNode * previousEdge = nullptr;  
    while (edge) {  
        if (edge->otherVertex() == v2) { // Found!  
            if (previousEdge) {  
                previousEdge->setNext(edge->next());  
            } else {  
                m_edges[v1] = edge->next();  
            }  
            delete edge;  
            break;  
        }  
        previousEdge = edge;  
        edge = edge->next();  
    }  
}
```

- **Exercício 2:** implemente uma classe para representar um grafo utilizando lista de adjacências.

```
void print() {  
    for (vertex i=0; i < m_numVertices; ++i) {  
        EdgeNode * edge = m_edges[i];  
        while (edge) {  
            printf("(%d,%d) ", i, edge->otherVertex());  
            edge = edge->next();  
        }  
        printf("\n");  
    }  
}
```

- **Exercício 2:** implemente uma classe para representar um grafo utilizando lista de adjacências.

```
GraphAdjList g2(6);  
g2.addEdge(0, 1);  
g2.addEdge(0, 2);  
g2.addEdge(1, 3);  
g2.addEdge(1, 4);  
g2.addEdge(2, 4);  
g2.addEdge(3, 4);  
g2.addEdge(4, 5);  
g2.addEdge(4, 1);  
g2.print();
```

**Output:**

```
(0, 1) (0, 2)  
(1, 3) (1, 4)  
(2, 4)  
(3, 4)  
(4, 1) (4, 5)
```



- **Exercício 3:** dados dois grafos  $G = (V, E)$  e  $H = (V', E')$  com  $V = V'$ , crie um algoritmo que verifica se  $H$  é subgrafo de  $G$ .
  - Usando matriz de adjacência:

```
bool isSubGraph(GraphMatrix & h) {  
    bool ** hEdges = h.edges();  
    for (vertex i=0; i < m_numVertices; i++) {  
        for (vertex j=0; j < m_numVertices; j++) {  
            if (hEdges[i][j]) { // Is there is an edge in H  
                if (!m_edges[i][j]) { // Check if it exists in G  
                    return false;  
                }  
            }  
        }  
    }  
    return true;  
}  
  
bool ** edges() { return m_edges; }
```

- Qual é a complexidade?

- **Exercício 3:** dados dois grafos  $G = (V, E)$  e  $H = (V', E')$  com  $V = V'$ , crie um algoritmo que verifica se  $H$  é subgrafo de  $G$ .
  - Usando lista de adjacência:

```
bool isSubGraph(GraphAdjList & h) {
    EdgeNode ** hEdges = h.edges();
    for (vertex i=0; i < m_numVertices; i++) { // For each vertex
        EdgeNode * hEdge = hEdges[i];
        while (hEdge) { // For each edge in H
            EdgeNode * gEdge = m_edges[i];
            bool found = false;
            while (gEdge) { // Check if it exists in G
                if (hEdge->otherVertex() == gEdge->otherVertex()) {
                    found = true;
                    break;
                }
                gEdge = gEdge->next();
            }
            if (!found) { return false; }
            hEdge = hEdge->next();
        }
    }
    return true;
}
```

- Qual é a complexidade?
- Como a solução poderia ser otimizada?

- **Exercício 4 (para fazer em casa):** dado um grafo  $G = (V, E)$  e um caminho  $P$  composto por uma sequência de vértices, verifique se  $P$  é um caminho de  $G$ , e se o caminho é simples.

# Perguntas?

`thiago.p.araujo@fgv.br`